



Specifica Tecnica

Gruppo Coderius

`coderius01@gmail.com`

Versione 0.7.3

Tabella di versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.7.3	2026/09/08	Leonardo Lorenzin	Ines Iadadi	Aggiornamento sezione 3.6
0.7.2	2026/09/08	Giovanni Bronte	Ines Iadadi	Correzione refusi e aggiunte minori
0.7.1	2026/09/04	Edis Hodja	Ines Iadadi	Revisione dell'architettura e correzione di refusi tecnici
0.7.0	2026/09/01	Alberto Canavese	Edis Hodja	Stesura sezione 7: «Requisiti di sistema»
0.6.0	2026/09/01	Ines Iadadi	Edis Hodja	Modifica alla struttura del documento e aggiornamento della sezione 2
0.5.0	2026/08/28	Alberto Canavese	Leonardo Lorenzin	Stesura della sezione 4 - Design pattern
0.4.3	2026/08/22	Ines Iadadi	Giovanni Bronte	Stesura della sezione Backend 3.6
0.4.2	2026/08/13	Filippo Zonta Rocha		Stesura della sezione 3.5
0.4.1	2026/08/10	Edis Hodja		Stesura della sezione 3.4
0.4.0	2026/07/29	Leonardo Lorenzin	Edis Hodja	Stesura iniziale della sezione 3
0.3.0	2026/07/24	Giovanni Bronte	Leonardo Lorenzin	Stesura della sezione 2
0.2.0	2026/07/24	Alberto Canavese	Leonardo Lorenzin	Stesura della sezione 1
0.1.0	2026/07/21	Giovanni Bronte	Leonardo Lorenzin	Prima stesura del documento

Indice

1. Introduzione	1
1.1. Scopo del documento	1
1.2. Scopo del prodotto	1
1.3. Glossario	3
1.4. Riferimenti	3
1.4.1. Riferimenti normativi	3
1.4.2. Riferimenti informativi	3
1.4.3. Documentazione delle tecnologie:	3
2. Tecnologie	4
2.1. Linguaggi di programmazione	4
2.2. Framework	4
2.3. Librerie	4
2.4. Strumenti di build e sviluppo	6
2.5. Persistenza dei dati	6
2.6. Deployment	7
2.7. Tecnologie per testing	8
3. Architettura del sistema	8
3.1. Architettura generale	8
3.1.1. Architettura logica	8
3.1.2. Modalità di comunicazione	9
3.1.3. Architettura di deployment	9
3.2. Stile architetturale	10
3.3. Pattern architetturali adottati	10
3.3.1. Client-Server	10
3.3.2. Layered Architecture	10
3.3.3. REST stateless	11
3.3.4. Dependency Injection	11
3.4. Motivazioni della scelta architetturale	11
3.4.1. Architettura a livelli anziché esagonale	11
3.4.2. Ripartizione delle responsabilità secondo il possesso del dato	12
3.4.3. Assenza di una base di dati	12
3.4.4. Interfaccia REST stateless	12
3.4.5. Monolite containerizzato anziché microservizi	12
3.5. Limiti dell'architettura	13
3.6. Design pattern	14
3.6.1. Criteri di scelta	14
3.6.2. Pattern architetturali	14
3.6.3. Pattern di progettazione adottati	15
3.6.4. Tabella riassuntiva	20

4. Decomposizione del sistema	22
4.1. Diagramma dei package	22
4.2. Frontend	22
4.2.1. Organizzazione a livelli	22
4.2.2. Presentation Layer	22
4.2.3. Presentation Layer	22
4.2.4. Application Layer	24
4.2.5. State Management	24
4.2.6. Infrastructure Layer	25
4.2.7. Domain Layer	26
4.2.8. Flussi applicativi	26
4.2.9. Caratteristiche di qualità	27
4.2.10. Elementi principali	27
4.3. Backend	35
4.3.1. Organizzazione a livelli	36
4.3.2. Presentation Layer	36
4.3.3. Application Layer	36
4.3.4. Domain Layer	36
4.3.5. Persistence Layer	37
4.3.6. Endpoint REST	37
4.3.7. Elementi principali	37
4.3.8. Testing	41
5. Diagrammi delle classi	41
5.1. Criteri di rappresentazione	41
5.2. Dominio frontend	41
5.2.1. Device	41
5.2.2. Asset	42
5.2.3. DecisionTree	43
5.2.4. Node, QuestionNode e LeafNode	44
5.2.5. Session	45
5.3. Dominio backend	47
5.3.1. AssetType	47
5.3.2. Device	47
5.3.3. Asset	48
5.3.4. NodeOutcome	48
5.3.5. Branches	48
5.3.6. Node, QuestionNode e LeafNode	49
5.3.7. DecisionTree	49
5.3.8. PathStep, Evaluation, Current e Session	50
5.4. Corrispondenza fra i due domini	51
5.5. Vincoli e regole di integrità	51

6. Tracciamento dei requisiti	53
6.1. Requisiti funzionali obbligatori	53
6.2. Requisiti funzionali desiderabili	57
6.3. Requisiti funzionali opzionali	59
7. Requisiti di sistema	61
7.1. Requisiti hardware	61
7.2. Requisiti software	62
7.3. Requisiti di rete	62

Elenco delle Figure

Figura 1 Diagramma delle pagine	23
---------------------------------------	----

1. Introduzione

1.1. Scopo del documento

Il presente documento descrive l'architettura logica e infrastrutturale del sistema sviluppato dal gruppo Coderius per il [Capitolato C1 - Automated EN18031 Compliance Verification, Bluewind S.r.l.](#)

Nel dettaglio, gli obiettivi preposti per questo documento sono:

- **Motivare l'infrastruttura tecnologica:** argomentare le ragioni dietro la scelta degli strumenti di sviluppo, selezionati per rispondere appieno ai criteri di flessibilità, performance e scalabilità richiesti dal progetto.
- **Strutturare la logica applicativa e i design pattern:** mappare l'organizzazione interna dei moduli e i loro flussi di comunicazione, dimostrando come i pattern implementati favoriscano la creazione di codice disaccoppiato e orientato al testing.
- **Pianificare la strategia di deployment:** definire le modalità di distribuzione dell'applicativo e l'allocazione delle sue parti all'interno degli ambienti operativi.
- **Promuovere la manutenibilità a lungo termine:** fornire direttive chiare al team di sviluppo per rendere la base di codice facilmente interpretabile, supportando così agevolmente le future iterazioni, correzioni ed espansioni del sistema.

1.2. Scopo del prodotto

Il prodotto mira ad automatizzare e digitalizzare il processo di verifica della conformità dei dispositivi radio alla normativa di sicurezza EN 18031, come richiesto dal capitolato

Automated EN18031 Compliance Verification proposto da Bluewind S.r.l.

L'obiettivo è sostituire le attuali procedure manuali, spesso dispendiose in termini di tempo e soggette ad errore umano, con una soluzione software moderna ed efficiente.

Le funzioni principali offerte dal prodotto sono le seguenti:

- **Definizione del dispositivo:** il sistema consente di descrivere il dispositivo da valutare e gli asset che lo compongono, creandoli tramite interfaccia web oppure importandoli da file nei formati JSON e CSV. Per ciascun asset il sistema deriva automaticamente i requisiti della norma applicabili sulla base della sua tipologia.
- **Esecuzione dei decision tree:** per ogni coppia asset-requisito il sistema esegue l'albero decisionale corrispondente, riconducendo l'interazione ad una sequenza di domande a risposta binaria affiancata dalla rappresentazione grafica dell'albero, che evidenzia il nodo corrente e il percorso già intrapreso.
- **Gestione della sessione di valutazione:** lo stato della valutazione può essere salvato su file in formato JSON in qualsiasi momento e ripreso successivamente dal punto in cui era stato interrotto.

- **Consultazione ed esportazione degli esiti:** gli esiti dei singoli requisiti sono aggregati a livello di asset e di dispositivo e restano risalibili fino alla sequenza di domande e risposte che li ha determinati. Il sistema produce inoltre un report di conformità finale in formato PDF, contenente per ciascuna coppia asset-requisito l'esito e il percorso logico seguito.
- **Gestione del catalogo dei decision tree:** il sistema mette a disposizione un catalogo degli alberi decisionali disponibili, consultabili in forma grafica, ampliabile importando nuovi alberi in formato JSON o CSV ed esportabile nei medesimi formati.

Il prodotto è concepito come strumento interno alla proponente, opera interamente in locale e non prevede autenticazione né archiviazione centralizzata. Lo scambio dei dati fra installazioni viene affidato all'esportazione e all'importazione di file.

1.3. Glossario

Al fine di evitare ambiguità e incomprensioni legate alla terminologia tecnica presente in questo documento e nel resto della documentazione prodotta dal gruppo Coderius, è stato redatto un apposito glossario.

Ogni termine tecnico o di dominio che necessita di ulteriori chiarimenti è contrassegnato nel testo da una «G» a pedice (es. *termine_G*). Cliccando sull'indicatore, il lettore verrà reindirizzato direttamente alla definizione corrispondente all'interno del documento

[Glossario](#).

1.4. Riferimenti

1.4.1. Riferimenti normativi

- [Capitolato C1 - Automated EN18031 Compliance Verification, Bluewind S.r.l.](#)
- [Norme di Progetto - Coderius Group](#)
- [Slide IS A.A. 2025/2026 - Regolamento del progetto didattico](#)
- [Norma EN 18031 — Common security requirements for radio equipment](#)

1.4.2. Riferimenti informativi

- [Slide IS A.A. 2023/2024 - Diagrammi delle Classi](#)
- [Slide IS A.A. 2022/2023 - Pattern Architetturali](#)
- [Slide IS A.A. 2022/2023 - Design Pattern Creazionali](#)
- [Slide IS A.A. 2022/2023 - Design Pattern Strutturali](#)
- [Slide IS A.A. 2025/2026 - Progettazione Software](#)

1.4.3. Documentazione delle tecnologie:

- [Python 3 Official Documentation](#)
- [TypeScript Official Documentation](#)
- [React Official Documentation](#)
- [Vite Getting Started Guide](#)
- [Flask Official Documentation](#)
- [Docker Documentation](#)
- [Vitest Documentation](#)
- [React Testing Library Documentation](#)

2. Tecnologie

Nella seguente sezione vengono descritte le tecnologie usate per lo sviluppo del capitolato **Automated EN18031 Compliance Verification**.

2.1. Linguaggi di programmazione

Tecnologia	Versione	Descrizione
Python	3.12	Linguaggio ad alto livello, interpretato e multi-paradigma, impiegato per la realizzazione del backend. La scelta risponde a un vincolo del capitolato, essendo il linguaggio prediletto dalla proponente, e trova conferma nella ricchezza della libreria standard e nella maturità dell'ecosistema di test.
TypeScript	6.0.2	Metalinguaggio open source sviluppato da Microsoft che si basa su JavaScript, aggiungendovi la tipizzazione statica. Nel nostro progetto è utilizzato per lo sviluppo del frontend assieme alla libreria React: l'introduzione dei tipi garantisce maggiore robustezza, facilita la manutenibilità del codice nel tempo e permette di intercettare numerosi errori già in fase di compilazione.

2.2. Framework

Tecnologia	Versione	Descrizione
Flask	3.1.3	Micro-framework Web scritto in Python, è progettato per essere rapido e semplice, offrendo al contempo la possibilità di essere utilizzato per applicazioni complesse. È inoltre molto semplice da imparare, flessibile e ha buone performance.
Flask-CORS	6.0.5	Estensione di Flask che gestisce le intestazioni <i>Cross-Origin Resource Sharing</i> . È necessaria in ambiente di sviluppo, dove client e server sono serviti da due origini distinte.

2.3. Librerie

Tecnologia	Versione	Descrizione
React	19.2.8	Libreria JavaScript per la costruzione di interfacce utente tramite componenti dichiarativi. Nel progetto costituisce la base del frontend, insieme a React Router, Zustand e alle altre librerie dell'ecosistema.
React Router	7.18.2	Libreria di instradamento lato client per React. Associa a ciascuna vista un percorso proprio e consente di applicare guardie di accesso alle pagine che richiedono precondizioni, come l'esistenza di una sessione di valutazione attiva.

React Flow	12.11	Libreria per la rappresentazione interattiva di grafi e diagrammi a nodi in applicazioni React, distribuita nel pacchetto <code>@xyflow/react</code> . Nel prodotto è impiegata per la visualizzazione del decision tree, con i nodi resi come componenti React personalizzati e supporto nativo a spostamento e ingrandimento della vista. La libreria non determina la disposizione dei nodi, che resta affidata alla regola pura <code>treeLayout</code> del Domain Layer: riceve le posizioni già calcolate e si occupa della sola resa grafica. Poiché la rappresentazione è di sola consultazione, spostamento e connessione dei nodi sono disabilitati.
React PDF	4.9	Libreria per la generazione di documenti PDF a partire da componenti React, distribuita nel pacchetto <code>@react-pdf/renderer</code> . Nel prodotto è impiegata per la produzione del report di conformità finale, la cui struttura è dichiarata come componente <code>ReportDocument</code> e resa in un documento scaricabile dall'utente. La generazione avviene nel client, utilizzando il dispositivo, la sessione e i decision tree resi disponibili dal catalogo.
Zustand	5.0	Libreria minimale per la gestione dello stato globale in applicazioni React. Gli store si definiscono come funzioni hook, senza richiedere componenti <i>Provider</i> né il <i>boilerplate</i> di azioni e <i>reducer</i> tipico di altre soluzioni. Nel prodotto realizza i tre store separati per area funzionale (<code>DeviceStore</code> , <code>SessionStore</code> , <code>TreeStore</code>).
Zod	4.4	Libreria per la definizione e validazione di schemi dati, con inferenza automatica dei tipi TypeScript a partire dallo schema stesso. Nel nostro progetto viene utilizzata nel Domain Layer del frontend per validare la struttura dei dati in ingresso (dispositivi, asset) prima che vengano elaborati dal resto dell'applicazione.
react-hot-toast	2.6	Libreria leggera per la gestione di notifiche toast in applicazioni React, con supporto nativo a coda, timer di auto-dismiss e stacking dei messaggi. Nel nostro progetto viene utilizzata nell'Infrastructure Layer del frontend come implementazione concreta dietro l'interfaccia <code>NotificationService</code> .
TanStack Query	5.101	Libreria per la gestione dello stato di origine remota in applicazioni React, con supporto a memorizzazione dei risultati, deduplicazione delle richieste in volo e ritentativi. Nel prodotto è impiegata in modo imperativo tramite

		queryClient, senza montare alcun <i>provider</i> , per evitare che lo stesso decision tree venga richiesto più volte nel corso di una valutazione.
--	--	--

2.4. Strumenti di build e sviluppo

Tecnologia	Versione	Descrizione
Vite	8.2.0	Vite è diventato lo standard de facto dell'ecosistema frontend. È il motore ufficiale dietro i framework moderni come Vue, e i template ufficiali di React. È inoltre molto più veloce a creare il server rispetto ad altri strumenti simili ed è anche molto facile da implementare grazie alla sua configurazione minima richiesta.
Node.js	22	Ambiente di esecuzione JavaScript, impiegato per l'esecuzione degli strumenti di sviluppo e per la fase di build dell'immagine Docker del frontend.
ESLint	10.8	Analizzatore statico per JavaScript e TypeScript, configurato con typescript-eslint per l'applicazione delle regole di codifica stabilite dalle Norme di Progetto. È eseguito automaticamente nella pipeline di integrazione continua.
Ruff	0.16.6	Analizzatore statico e formattatore per Python. Nel prodotto è configurato con una lunghezza massima di riga di 100 caratteri ed è eseguito in integrazione continua nelle modalità <code>ruff check</code> e <code>ruff format --check</code> .

2.5. Persistenza dei dati

Il sistema non impiega alcun sistema di gestione di basi di dati. La scelta è coerente con la natura del dominio applicativo: l'unica collezione di dati posseduta dal sistema è il catalogo dei decision tree, un insieme ristretto di documenti strutturati, di sola lettura durante l'esecuzione di una valutazione e privo di relazioni da interrogare.

Tecnologia	Versione	Descrizione
JSON	RFC 8259	Formato di scambio dati impiegato in tre ruoli distinti: rappresentazione dei decision tree nella cartella backend/data/decision_trees/, corpo delle richieste e delle risposte dell'API REST, e formato di esportazione e importazione di dispositivi e sessioni di valutazione.
CSV	RFC 4180	Formato tabellare impiegato come formato alternativo di scambio per i dispositivi e per i decision tree, secondo quanto richiesto dai requisiti RF-Ob05, RF-Ob21, RF-Ob79 e RF-D23.

Il dispositivo in lavorazione e la sessione di valutazione non sono mai scritti su disco dal server: risiedono nello stato del client per l'intera durata del lavoro e vengono esportati su

file su richiesta dell'utente. Le implicazioni architetture di questa scelta sono discusse nella sezione Sezione 3.

2.6. Deployment

Tecnologia	Versione	Descrizione
Docker	28.5.1	Piattaforma open source che consente agli sviluppatori di creare, implementare, eseguire, aggiornare e gestire i container. I container sono componenti standardizzati ed eseguibili che combinano il codice sorgente dell'applicazione con le librerie e le dipendenze del sistema operativo necessarie per eseguire tale codice in qualsiasi ambiente.
Docker Compose	2.40.0	Strumento di orchestrazione dei container. Il file <code>docker-compose.yml</code> descrive i due servizi, la rete privata che li collega, l'esposizione delle porte verso l'ospite (8080 per il frontend, 5000 per il backend) e il montaggio della cartella dei decision tree come volume.
Nginx	1.27	Server HTTP impiegato nell'immagine del frontend per servire i file statici prodotti dalla build e per inoltrare al backend le richieste dirette agli endpoint dell'API. La presenza del <i>reverse proxy</i> consente al client di rivolgersi a percorsi relativi, rendendo indirizzo e porta del backend un dettaglio della configurazione di deployment anziché del codice applicativo.

2.7. Tecnologie per testing

Tecnologia	Versione	Descrizione
Vitest	4.1.10	Vitest è un framework per il test di applicazioni TypeScript e JavaScript, progettato per essere veloce e leggero. Sfruttando la stessa configurazione di Vite, garantisce un ambiente di testing altamente performante e coerente con la build.
React Testing Library	16.3.2	Libreria per il testing di componenti React. Offre funzioni di utilità essenziali basate su react-dom e react-dom/test-utils, incoraggiando al contempo migliori pratiche di testing.
Pytest	9.1.1	Framework per il test di Python, che permette di scrivere test automatizzati in modo semplice e leggibile.

3. Architettura del sistema

3.1. Architettura generale

3.1.1. Architettura logica

Il sistema adotta il paradigma **Client-Server**, con una separazione netta fra la componente che gestisce l'interazione con l'utente e quella che ospita la logica applicativa e il possesso dei dati. Tenere distinte le due responsabilità consente di far evolvere ciascuna parte in modo autonomo e riduce le dipendenze reciproche fra le tecnologie impiegate.

Il prodotto si compone di due unità distribuibili:

- un **client** React, che costituisce l'intera interfaccia grafica: presenta le viste, raccoglie e valida gli input, conserva lo stato della valutazione in corso, esegue la navigazione lungo il decision tree e produce i file di esportazione;
- un **server** Flask scritto in Python 3.12, che espone un'API REST: valida i dati in ingresso, possiede il catalogo dei decision tree e ne governa il caricamento, l'importazione, l'esportazione e la derivazione dei requisiti applicabili ad un tipo di asset.

La ripartizione delle responsabilità fra le due unità segue il criterio del possesso del dato. Il server possiede il catalogo dei decision tree, che mette a disposizione dell'utente; non possiede invece né il dispositivo né la sessione di valutazione, che l'utente porta al sistema e riporta via sotto forma di file. Ne consegue che la navigazione lungo l'albero avviene interamente nel client; rispondere ad una domanda del decision tree non comporta alcuna richiesta al server, poiché l'albero è stato scaricato per intero all'ingresso nel requisito.

Al server ci si rivolge esclusivamente per ottenere un decision tree o l'elenco del catalogo, importare o esportare un decision tree, risolvere gli identificativi di dispositivo e asset e derivare i requisiti applicabili ad un tipo di asset. Ogni altra operazione, come

l'avanzamento nell'albero, calcolo degli esiti aggregati, salvataggio e ripresa della sessione, esportazione del dispositivo, viene svolta nel client.

Internamente ciascuna unità è organizzata a livelli con dipendenza a senso unico.

La collocazione del report nel client si fonda unicamente su dispositivo, sessione di valutazione e alberi dei requisiti esaminati, dati che risiedono nel client. Produrlo lato server avrebbe richiesto di trasmettere l'intera sessione a un componente che non la possiede, introducendo uno stato applicativo in contrasto con il carattere stateless dell'API. Non è stato pertanto introdotto alcun endpoint dedicato.

3.1.2. Modalità di comunicazione

Client e server dialogano esclusivamente attraverso richieste **HTTP** secondo lo stile architetturale **REST**, con i dati veicolati in formato JSON. Fa eccezione l'importazione di un decision tree, che trasmette il file come multipart/form-data, e la sua esportazione, che restituisce text/csv quando richiesto in tale formato.

Lo scambio è stateless in quanto il server non conserva alcuno stato di sessione fra una richiesta e la successiva, e ogni chiamata contiene tutte le informazioni necessarie alla propria elaborazione. Questa impostazione è dovuta al fatto che dispositivo e sessione risiedono nel client e di conseguenza non vi è alcuno stato conversazionale da mantenere lato server.

Il client non conosce l'indirizzo del server. `FetchApiClient` è costruito con un `baseUrl` vuoto e compone quindi percorsi relativi (`/devices`, `/assets`, `/decision-trees`), che raggiungono il backend grazie a un **reverse proxy**:

- in sviluppo è il server di Vite, la cui configurazione inoltra i percorsi dell'API a `http://localhost:5000`;
- in produzione è Nginx, che nell'immagine del frontend serve i file statici della build e inoltra i medesimi percorsi al servizio backend sulla rete privata dei container.

L'indirizzo e la porta del backend sono pertanto un dettaglio della configurazione di deployment e non compaiono nel codice applicativo. L'estensione `Flask-CORS` resta necessaria per gli scenari di esecuzione in cui client e server sono serviti da origini distinte.

Il client ignora completamente il modo in cui i dati sono archiviati sul server e il modo in cui la validità di un albero viene verificata: si attiene ai contratti stabiliti dagli endpoint, che costituiscono l'unico punto di contatto fra le due componenti e l'unica fonte di verità condivisa.

3.1.3. Architettura di deployment

Il sistema è distribuito come applicazione **monolitica containerizzata**: un solo servizio di backend e un solo servizio di frontend, orchestrati tramite Docker Compose e collegati da una rete privata.

I due servizi sono così configurati:

- **backend:** immagine derivata da `python:3.12-slim`, espone la porta 5000 e monta la cartella `backend/data` come volume, cosicché i decision tree importati dall'utente persistano fra i riavvii del container;
- **frontend:** immagine costruita in due fasi che espone la porta 80 del container sulla porta 8080 dell'ospite.

La scelta di un monolite in luogo di una scomposizione in microservizi è coerente con la natura del dominio applicativo: il sistema elabora un dispositivo alla volta all'interno di una singola sessione di valutazione, per un solo utente, senza richiedere né elaborazione concorrente di grandi volumi di richieste né scalabilità orizzontale indipendente delle parti. L'adozione di microservizi introdurrebbe una complessità infrastrutturale sproporzionata a fronte di un dominio ben delimitato.

La separazione fra client e server, unita all'organizzazione interna a livelli, è già sufficiente a garantire modularità e disaccoppiamento senza ricorrere a una scomposizione fisica. Un monolite containerizzato risulta inoltre più semplice da avviare e arrestare, richiede meno risorse computazionali e consente cicli di rilascio più rapidi: caratteristiche adatte sia alle dimensioni del gruppo, sia al carattere di strumento interno del prodotto.

3.2. Stile architetturale

Entrambe le unità adottano internamente un'**architettura a livelli** (*Layered Architecture*), con dipendenza a senso unico dai livelli superiori verso quelli inferiori. Il livello di dominio non dipende da alcun altro livello.

Si è deliberatamente evitata l'applicazione rigorosa di un'architettura esagonale. Il dominio applicativo è delimitato ad un solo meccanismo di persistenza su filesystem, una sola API REST esposta a un solo tipo di client. Introdurre porte e adattatori espliciti per ogni dipendenza tecnica aggiungerebbe indirezionalità senza un beneficio proporzionato in termini di sostituibilità. Il disaccoppiamento esplicito fra interfaccia e implementazione è adottato là dove esiste una ragione concreta per prevedere un'implementazione alternativa (l'accesso al catalogo dei decision tree, la comunicazione HTTP, la presentazione delle notifiche) e omissis altrove.

Analogamente, lato client gli hook applicativi accedono direttamente agli store anziché passare da un'astrazione intermedia, mantenendo comunque il vincolo di dipendenza a senso unico.

3.3. Pattern architetturali adottati

3.3.1. Client-Server

Il pattern è realizzato dalla separazione fra il client React e il server Flask, eseguiti come container distinti e comunicanti tramite HTTP.

3.3.2. Layered Architecture

L'organizzazione interna di entrambe le unità in livelli con dipendenza a senso unico è il vincolo strutturale portante del prodotto. È ciò che permette di sostituire un dettaglio

tecnico senza toccare la logica applicativa, ed è verificabile: i test del dominio non richiedono alcun contesto applicativo Flask né alcun componente React montato per essere eseguiti. La composizione dei livelli è descritta in Sezione 4.

3.3.3. REST stateless

Il server non mantiene stato di sessione fra le richieste. Ogni chiamata è autosufficiente, il che rende le rotte verificabili in isolamento e rimuove ogni vincolo di ordine fra le richieste.

3.3.4. Dependency Injection

Il principio prevede che un componente riceva i propri collaboratori dall'esterno anziché istanziarli, cosicché il grafo degli oggetti sia composto in un unico punto del sistema.

Lato **backend** il principio è applicato integralmente: `create_app()` è l'unico punto in cui compaiono i nomi delle classi concrete, e costruisce il repository, lo inietta nel `DecisionTreeService` e passa quest'ultimo alle factory dei blueprint. La direzione delle dipendenze è pertanto verificabile ispezionando un solo file.

Lato **frontend** l'applicazione è parziale, e costituisce il principale debito architetturale individuato:

- `DeviceService` istanzia direttamente `FetchApiClient` a livello di modulo;
- `DecisionTreeService` accetta un'implementazione di `ApiClientService` nel costruttore, ma ne dichiara una concreta come valore predefinito;
- `DecisionTreeService.exportTree()` invoca `fetch` direttamente, eludendo l'interfaccia `ApiClientService`.

Le prime due condizioni non compromettono la verificabilità ma introducono una dipendenza in fase di compilazione dalla classe concreta, in violazione del principio di inversione delle dipendenze. La terza è una violazione della regola secondo cui ogni comunicazione con il backend transita per `ApiClientService`.

3.4. Motivazioni della scelta architetturale

Le decisioni architetturali descritte nelle sezioni precedenti sono qui raccolte con la rispettiva motivazione, così da rendere esplicito il ragionamento che le ha prodotte e i vincoli entro cui sono state prese. Ciascuna è stata valutata rispetto alle alternative disponibili, non adottata per consuetudine.

3.4.1. Architettura a livelli anziché esagonale

L'architettura esagonale avrebbe imposto la definizione di porte e adattatori espliciti per ogni dipendenza tecnica. Il dominio applicativo è però delimitato: un solo meccanismo di persistenza, una sola API esposta a un solo tipo di client, nessuna previsione di sorgenti dati alternative. In tali condizioni l'introduzione sistematica di astrazioni aggiungerebbe indirezione senza variabilità da governare.

L'architettura a livelli garantisce gli stessi due benefici che qui interessano — separazione delle responsabilità e verificabilità del dominio in isolamento — a un costo strutturale

inferiore. Il disaccoppiamento esplicito fra interfaccia e implementazione è stato adottato in modo **selettivo**, nei tre punti in cui esiste una ragione concreta per prevedere una realizzazione alternativa: l'accesso al catalogo dei decision tree, la comunicazione HTTP e la presentazione delle notifiche.

3.4.2. Ripartizione delle responsabilità secondo il possesso del dato

La ripartizione fra client e server non segue la separazione convenzionale fra presentazione e logica di business, ma il criterio del possesso del dato: il server ospita ciò che il sistema possiede e mette a disposizione dell'utente, il client ciò che l'utente porta al sistema e riporta via.

Il catalogo dei decision tree è dato del sistema; il dispositivo e la sessione di valutazione sono dell'utente, che li crea, li esporta su file e li reimporta. Ne discende che la navigazione lungo l'albero avviene interamente nel client e che il server non conserva né dispositivi né sessioni.

Il criterio produce due conseguenze verificabili: nessuna richiesta di rete è necessaria per rispondere a una domanda del decision tree, e l'utente conserva il pieno controllo dei propri dati, che non vengono persistiti né conservati in un archivio condiviso dal server.

Quest'ultima proprietà è coerente con la natura di strumento interno del prodotto e con la decisione VE-7.1.

3.4.3. Assenza di una base di dati

L'unica collezione posseduta dal sistema è il catalogo dei decision tree: un insieme ristretto di documenti strutturati, di sola lettura durante una valutazione e privo di relazioni da interrogare. Un sistema di gestione di basi di dati risolverebbe problemi che il prodotto non presenta — interrogazione relazionale, accesso concorrente in scrittura, integrità referenziale — introducendo un terzo servizio da avviare e configurare su ciascuna postazione.

La persistenza su file JSON mantiene il catalogo ispezionabile e modificabile con strumenti ordinari, proprietà utile in un contesto in cui gli alberi sono scambiati manualmente fra installazioni.

3.4.4. Interfaccia REST stateless

L'assenza di stato conversazionale lato server non è una scelta indipendente, ma la conseguenza diretta del criterio di possesso del dato: poiché dispositivo e sessione risiedono nel client, non esiste alcuno stato da mantenere fra una richiesta e la successiva. Ne deriva che ogni rotta è verificabile in isolamento e che non sussiste alcun vincolo di ordine fra le chiamate.

3.4.5. Monolite containerizzato anziché microservizi

Il sistema elabora un dispositivo alla volta, per un solo utente, senza elaborazione concorrente di grandi volumi né necessità di scalare le parti in modo indipendente. Una scomposizione in servizi introdurrebbe comunicazione fra servizi, individuazione dei servizi e coerenza dei dati distribuiti: complessità infrastrutturale priva di contropartita.

La separazione fra client e server, unita all'organizzazione interna a livelli, fornisce già la modularità necessaria senza richiedere una scomposizione fisica. La containerizzazione garantisce inoltre che l'ambiente di esecuzione sia il medesimo su ogni postazione, requisito rilevante per un prodotto avviato localmente da utenti che non sono sviluppatori.

3.5. Limiti dell'architettura

Le scelte descritte comportano limiti che si ritiene corretto dichiarare. Nessuno di essi compromette il soddisfacimento dei requisiti, ma ciascuno delimita il campo entro cui il prodotto può evolvere senza una revisione architetturale.

Il vincolo di dipendenza fra livelli è una disciplina, non una proprietà imposta. Né Flask né React impediscono a un modulo di un livello superiore di invocare direttamente un meccanismo tecnico: nulla vieta a una pagina di chiamare `fetch`. Il vincolo è presidiato dalle convenzioni di codifica e dalla revisione fra pari, non dagli strumenti. Che si tratti di un limite reale è dimostrato dal caso di `DecisionTreeService.exportTree()`, che invoca `fetch` eludendo l'interfaccia `ApiClientService`.

La logica di valutazione risiede nel client. La navigazione dell'albero, il calcolo degli esiti aggregati e la gestione della sessione sono realizzati in TypeScript e non sono raggiungibili dall'esterno. L'introduzione di un secondo tipo di client — un'applicazione mobile, un'interfaccia a riga di comando, un'integrazione con sistemi della proponente — richiederebbe di reimplementare tale logica oppure di spostarla lato server, con una revisione del criterio di possesso del dato.

Il modello di dominio è duplicato in due linguaggi. Le entità esistono in TypeScript e in Python senza generazione automatica dell'una dall'altra: la coerenza è presidiata unicamente dal contratto degli endpoint e dai test. Una divergenza fra le due rappresentazioni non è rilevata da alcuno strumento in fase di compilazione.

L'assenza di base di dati esclude l'interrogazione trasversale. Ogni operazione che percorra l'intero catalogo — come la derivazione dei requisiti applicabili a un tipo di asset — richiede la lettura di tutti i file. Il costo è trascurabile con alcune decine di documenti, ma l'introduzione di funzionalità di ricerca o di filtraggio sul catalogo imporrebbe di riconsiderare la decisione.

L'assenza di persistenza dei dati dell'utente sposta su di lui la responsabilità della conservazione. Un dispositivo o una sessione non esportati su file vanno perduti alla chiusura dell'applicazione. È una conseguenza accettata del criterio di possesso del dato e della decisione VE-7.1, non un difetto di realizzazione, ma va tenuta presente nella redazione del Manuale Utente.

Il monolite non consente scalabilità indipendente delle parti. La condizione è irrilevante nello scenario d'uso previsto, in cui un'installazione serve un solo utente per volta, ma lo diverrebbe qualora il prodotto fosse esposto come servizio condiviso.

3.6. Design pattern

3.6.1. Criteri di scelta

I pattern documentati in questa sezione non sono stati selezionati a priori da un catalogo, ma individuati a partire dai problemi concretamente emersi nella progettazione del prodotto. Ciascuno di essi risponde ad almeno uno dei seguenti obiettivi, coerenti con l'organizzazione a livelli descritta nelle sezioni precedenti:

- **Verificabilità:** consentire il collaudo della logica applicativa senza richiedere una rete attiva, un'interfaccia grafica montata o file reali sul disco.
- **Assenza di logica applicativa ai bordi:** mantenere pagine e rotte prive di decisioni, in coerenza con il vincolo di dipendenza a senso unico fra i livelli.
- **Estendibilità rispetto ai requisiti EN 18031:** permettere l'introduzione di nuovi requisiti e di nuovi formati di scambio senza riprogettare i moduli esistenti.
- **Unicità del punto di verità:** evitare la duplicazione di logiche di validazione o di trasformazione dei dati in punti diversi del sistema.

Si sottolinea che nessuna scelta è motivata dalla previsione di un'infrastruttura distribuita. L'applicazione opera interamente in locale e la condivisione dei decision tree modificati avviene all'esterno dell'applicativo, mediante esportazione e distribuzione manuale dei file.

Il perimetro di questa sezione è costituito dai pattern del catalogo di Gamma, Helm, Johnson e Vlissides che presentano attinenza con il problema affrontato, integrati dal solo Repository, che appartiene al catalogo architetturale di Martin Fowler ed è documentato in quanto criterio di accesso ai dati adottato dal backend. I pattern del catalogo privi di rapporto con il dominio applicativo non sono discussi. Ciò comporta che alcune tecniche di progettazione ampiamente impiegate nel prodotto non vi compaiano come pattern: è il caso dell'iniezione delle dipendenze, per cui ciascun componente riceve i propri collaboratori dal costruttore anziché istanziarli, e il grafo degli oggetti viene composto in un unico punto del sistema. Si tratta di un principio di progettazione e non di un pattern del catalogo, e come tale è descritto contestualmente al Factory Method, che ne costituisce il luogo di applicazione, anziché in una scheda propria.

Per ciascun pattern sono indicati il problema affrontato, la soluzione adottata e i moduli concretamente coinvolti. Tutti i pattern documentati in questa sezione sono stati individuati in corrispondenza di meccanismi effettivamente presenti nel codice; la sezione si chiude con quelli che non trovano applicazione, corredati della motivazione tecnica che ne esclude l'adozione.

3.6.2. Pattern architetturali

I pattern architetturali che governano la struttura complessiva del sistema sono già stati motivati nelle sezioni precedenti e vengono qui richiamati per completezza:

- **Client-Server:** separazione tra il client React e il server Flask, entrambi eseguiti in locale come container distinti e coordinati da Docker Compose.
- **Layered Architecture:** organizzazione interna in livelli con dipendenza a senso unico, in cinque livelli lato frontend e quattro lato backend.
- **REST stateless:** assenza di stato di sessione lato server; ogni richiesta contiene tutte le informazioni necessarie alla propria elaborazione.
- **Monolite containerizzato:** distribuzione come singolo backend e singolo frontend, in luogo di una scomposizione in servizi indipendenti.

3.6.3. Pattern di progettazione adottati

3.6.3.1. Adapter

- **Problema:** la logica applicativa incontra in due punti un meccanismo tecnico la cui interfaccia è espressa nei termini della tecnologia e non in quelli del problema: la funzione `fetch` per il dialogo con il backend, la libreria di notifica per i messaggi all'utente. Se tali interfacce penetrassero nei service, ogni collaudo della logica applicativa richiederebbe un backend in esecuzione o un'interfaccia grafica montata, e la verifica dei casi di errore diventerebbe impraticabile.
- **Soluzione:** definire un'interfaccia stabile espressa nei termini del dominio applicativo e realizzarla con una classe che, per delega anziché per ereditarietà, ne traduce le chiamate nell'interfaccia incompatibile del meccanismo sottostante. Si adotta quindi la variante *Object Adapter*: gli elementi da adattare sono una funzione globale (`fetch`) e un modulo importato (*react-hot-toast*), non classi da cui una gerarchia possa derivare.
- **Applicazione nel progetto:** il pattern è applicato a due confini tecnici del sistema.
 - `FetchApiClient` realizza l'interfaccia `ApiClientService`, che dichiara le operazioni `get`, `post`, `postFormData` e `delete` tipizzate. Il metodo privato `request()` concentra la costruzione dell'indirizzo, l'interpretazione della risposta e la traduzione degli esiti: un fallimento di rete e una risposta non riuscita diventano entrambi un `ApiError`, che espone il messaggio e lo stato numerico quando disponibile.
 - `NotificationManager` realizza l'interfaccia `NotificationService`, delegando alla libreria *react-hot-toast* la gestione di coda, timer di scomparsa automatica e impilamento dei messaggi.
- **Conseguenze:** i test di `FetchApiClient` verificano il trattamento delle risposte di errore senza alcun backend in esecuzione. Ne consegue che ogni comunicazione con il backend deve transitare per `ApiClientService`. L'esportazione del decision tree, che allo stato attuale invoca `fetch` direttamente, dovrà essere ricondotta a tale regola.

3.6.3.2. Repository

- **Problema:** il catalogo dei decision tree è l'unico insieme di dati che il backend possiede e mette a disposizione dell'utente, e vi accede oggi leggendo file JSON dal filesystem. Se i service invocassero direttamente i moduli di accesso al filesystem, il criterio di reperimento dei dati risulterebbe disseminato in ogni punto che ne ha bisogno e la logica applicativa sarebbe collaudabile soltanto predisponendo file reali su disco a ogni esecuzione della suite, rendendo di fatto impraticabile la verifica dei casi di errore.
- **Soluzione:** interporre fra Application Layer e sorgente dei dati un'interfaccia che espone le operazioni sulla collezione di entità come se questa fosse disponibile in memoria, nascondendo al chiamante il meccanismo di accesso effettivo.
- **Applicazione nel progetto:** l'interfaccia astratta `IDecisionTreeRepository` dichiara i metodi `get()`, `save()`, `delete()` e `list()`; l'implementazione concreta `JsonDecisionTreeRepository` li realizza leggendo e scrivendo i file contenuti in `backend/data/decision_trees/`. Il `DecisionTreeService` riceve l'interfaccia nel costruttore e ignora completamente la natura della sorgente, tanto nel reperimento del singolo albero quanto nelle operazioni che percorrono l'intero catalogo, come la derivazione dei requisiti applicabili a un tipo di asset.
- **Conseguenze:** il beneficio è verificabile nel codice esistente, dove i test del `DecisionTreeService` sostituiscono l'implementazione concreta con la classe `FakeDecisionTreeRepository`, che realizza la medesima interfaccia mantenendo gli alberi in memoria e consente quindi di collaudare la logica applicativa senza alcun accesso al disco. L'interfaccia non è motivata dalla previsione di un archivio centralizzato bensì da questa verificabilità e dalla necessità, prevista dai casi d'uso di modifica del decision tree, di scrivere gli alberi su disco attraverso un unico punto di accesso.

3.6.3.3. Facade

- **Problema:** diverse operazioni che l'utente percepisce come unitarie sono in realtà sequenze articolate. L'importazione di un dispositivo da file richiede il riconoscimento del formato, la lettura asincrona tramite `FileReader`, l'interpretazione del contenuto, la validazione dello schema e infine l'invio al backend dei metadati del device e di ciascun asset. Esporre tale sequenza alle viste le legherebbe a dettagli estranei alla presentazione e ne impedirebbe il riuso da parte di viste diverse.
- **Soluzione:** raggruppare la sequenza dietro un'unica operazione di alto livello, espressa nel linguaggio del caso d'uso, che coordini internamente i collaboratori necessari.
- **Applicazione nel progetto:** lato client, `importDeviceFromFile(file)` racchiude l'intera sequenza di importazione e restituisce alla vista il dispositivo costruito insieme al payload originale. Analogamente, `exportReportPdf(session)` racchiude la produzione del report di conformità: la raccolta dei dati dalla sessione con il reperimento degli alberi necessari, la resa del documento PDF e la consegna del file all'utente, esponendo alla pagina una sola operazione asincrona. I custom hook assolvono la funzione analoga verso il Presentation

Layer: `useSessionRunner()` coordina le fasi della valutazione guidata, il caricamento e l'idratazione dell'albero e la registrazione dell'esito, esponendo alla pagina soltanto lo stato e le azioni necessarie; `useResult()` opera allo stesso modo per la consultazione degli esiti. Lato server, `DecisionTreeService.get_tree()` racchiude in una sola chiamata l'accesso al catalogo, la gestione dell'assenza del dato (`DecisionTreeNotFoundError`) e la normalizzazione dell'albero, mentre `create_device()` e `create_asset()` costituiscono il punto d'ingresso unico per la costruzione delle rispettive entità, condiviso dal percorso di creazione manuale e da quello di importazione: entrambi delegano la validazione ai metodi statici del dominio, e `create_asset()` vi aggiunge la sola risoluzione dei requisiti predefiniti, che richiede l'accesso al catalogo.

- **Conseguenze:** pagine e rotte restano prive di logica applicativa e si limitano, rispettivamente, a renderizzare e a deserializzare, delegare e serializzare. L'unicità del punto di validazione evita che due percorsi diversi applichino allo stesso concetto regole divergenti.

3.6.3.4. Factory Method

- **Problema:** i nodi di un decision tree provengono dai file di catalogo in forma piatta, e la classe da istanziare dipende dal dato ricevuto: un nodo che dichiara i propri rami è una domanda, uno che dichiara un esito è una foglia. Se la decisione fosse presa nei punti di utilizzo, ogni modulo che percorre l'albero dovrebbe conoscere entrambe le classi concrete e replicare il medesimo controllo, e l'introduzione di una nuova tipologia di nodo richiederebbe di intervenire in ciascuno di essi.
- **Soluzione:** concentrare in una funzione dedicata la decisione sulla classe concreta da istanziare, restituendo al chiamante un'istanza utilizzabile attraverso il solo contratto comune.
- **Applicazione nel progetto:** le funzioni `_normalize_node()` e `createNode()` ricevono il nodo grezzo e scelgono fra `QuestionNode` e `LeafNode` in base al campo `type`, restituendo un'istanza che i chiamanti impiegano attraverso le sole operazioni `next()` e `verdict()`. Il pattern è declinato nella variante parametrica, ovvero la classe concreta è determinata da un valore ricevuto anziché da una sottoclasse del creatore, coerentemente con i paradigmi modulari di TypeScript e Python, che non richiedono una gerarchia di fabbriche dedicate per ottenere il medesimo effetto.
- **Conseguenze:** i moduli che percorrono l'albero non ispezionano il tipo del nodo: in `treeRules.ts` la funzione `currentOutcome()` si limita a invocare `node.verdict()`, che restituisce l'esito per una foglia e nulla per una domanda. L'introduzione di una nuova tipologia di nodo, richiesta da un'eventuale evoluzione della norma, riguarderebbe la sola funzione di costruzione e la nuova classe. La costruzione delle restanti entità di dominio segue un criterio analogo ma senza scelta di tipo, attraverso i metodi statici descritti in Sezione 5, mentre la composizione del grafo degli oggetti applicativi è trattata in Sezione 3 come applicazione della Dependency Injection.

3.6.3.5. Observer

- **Problema:** lo stato condiviso fra più viste — il dispositivo in lavorazione, la sessione di valutazione, l'albero corrente — deve provocare l'aggiornamento dei soli componenti effettivamente interessati, senza che i moduli che modificano lo stato debbano conoscere i componenti che lo consumano.
- **Soluzione:** i consumatori si registrano presso il detentore dello stato, che notifica automaticamente ogni variazione ai soli osservatori interessati.
- **Applicazione nel progetto:** gli store Zustand DeviceStore, SessionStore e TreeStore costituiscono i soggetti osservati; i componenti si registrano tramite hook selettori, come `useSessionStore((state) => state.session)`, che circoscrivono la sottoscrizione alla sola porzione di stato utilizzata. Le modifiche avvengono unicamente attraverso le azioni tipizzate esposte da ciascuno store: `setDevice`, `updateDeviceDetails`, `addAsset`, `updateAsset` e `removeAsset` per DeviceStore; `start`, `ensureSession`, `resume`, `syncProgress`, `completeCurrent` e `select` per SessionStore; `loadTree`, `hydrate`, `answer`, `goBack` e `goForward` per TreeStore. Ciascuno store espone inoltre `reset`, che ne ripristina lo stato iniziale.
- **Conseguenze:** il flusso dei dati resta unidirezionale e ogni variazione di stato è riconducibile a un'azione esplicita e tracciabile. La suddivisione per area funzionale, in luogo di un unico store globale, estende allo stato condiviso la separazione delle responsabilità adottata a livello di moduli, mentre la granularità dei selettori evita i re-render indiscriminati. Gli store non sono del tutto indipendenti: DeviceStore invoca la reimpostazione di SessionStore quando il dispositivo viene sostituito o modificato, poiché una sessione di valutazione riferita a un dispositivo che non esiste più produrrebbe esiti privi di significato. Si tratta di una dipendenza deliberata e a senso unico, che realizza un vincolo di dominio anziché un accoppiamento accidentale.

3.6.3.6. Proxy

- **Problema:** il pattern risponde nel prodotto a due esigenze distinte, entrambe riconducibili al controllo dell'accesso a una risorsa. Da un lato l'albero decisionale di un requisito viene richiesto ripetutamente nel corso di una valutazione, mentre si tratta di dati di catalogo immutabili per l'intera durata della sessione. Dall'altro la pagina di esecuzione presuppone l'esistenza di una sessione attiva e non può essere raggiunta direttamente per indirizzo.
- **Soluzione:** interporre fra il chiamante e la risorsa un sostituto che ne espone la medesima interfaccia e ne governa l'accesso, aggiungendovi la conservazione del risultato oppure la verifica delle precondizioni.
- **Applicazione nel progetto:** la variante con verifica delle precondizioni è realizzata nella sua forma piena dal componente `RequireSession`, che avvolge la pagina di esecuzione della sessione: presenta al router la stessa interfaccia della pagina protetta e ne consente il rendering solo in presenza di una sessione, reindirizzando altrimenti alla pagina iniziale.

La variante con conservazione del risultato è invece delegata a *TanStack Query*: `DecisionTreeService` avvolge le proprie chiamate in `queryClient.fetchQuery()`, che restituisce il risultato già ottenuto per la medesima chiave anziché ripetere la richiesta. La configurazione dichiara `staleTime` e `gcTime` illimitati e disabilita i ritentativi, coerentemente con la natura immutabile del dato: la conservazione serve a non rileggere più volte lo stesso catalogo, non a compensare l'inaffidabilità della comunicazione. L'accesso alla rete continua ad avvenire tramite l'interfaccia `ApiClientService`, che il service riceve nel costruttore.

- **Conseguenze:** le pagine non contengono controlli di accesso e i service applicativi non contengono logica di conservazione dei risultati; entrambi i meccanismi sono rimovibili senza modificare il codice che governano. Le due applicazioni non hanno tuttavia lo stesso grado di aderenza al pattern: `RequireSession` ne realizza la struttura, essendo un sostituto interposto fra il router e la pagina e a questa sostituibile, mentre la conservazione dei risultati ne applica il principio senza la struttura, poiché `queryClient.fetchQuery()` è invocato all'interno dei metodi del service e non esiste una coppia di implementazioni intercambiabili dietro un'interfaccia comune. La libreria è inoltre utilizzata in modo imperativo e non attraverso i propri hook: l'applicazione non monta alcun provider, e il meccanismo resta pertanto confinato all'*Infrastructure Layer*.

3.6.3.7. Strategy

- **Problema:** due famiglie di dati devono poter essere scritte e rilette in formati diversi, oggi JSON e CSV: il dispositivo con i suoi asset e i decision tree del catalogo. La decisione VE-7.1 accresce il peso di questa funzionalità: poiché i decision tree modificati non si sincronizzano fra le installazioni ma vengono esportati e distribuiti manualmente, i file di scambio sono il canale con cui gli utenti si trasmettono il lavoro svolto. Governare la varietà dei formati con una struttura condizionale interna ai moduli di importazione ed esportazione comporterebbe di modificarli a ogni nuova esigenza, con conseguente rischio di regressione sui formati già funzionanti.
- **Soluzione:** definire una famiglia di algoritmi intercambiabili dietro un'interfaccia comune e selezionare a runtime l'implementazione corrispondente al formato richiesto.
- **Applicazione nel progetto:** il pattern è applicato due volte, su entrambi i lati del sistema.
 - Lato client, per il dispositivo: l'interfaccia `DeviceFileFormat` dichiara l'estensione, il tipo di contenuto e le due operazioni simmetriche `serialize(device)` e `parse(text)`; le implementazioni concrete `jsonDeviceFormat` e `csvDeviceFormat` le realizzano per i rispettivi formati, ciascuna con le proprie regole di interpretazione. La funzione `formatForFile(file)` seleziona la strategia in base al file fornito dall'utente; `importDeviceFromFile()` e `exportDevice()` la utilizzano senza conoscerne l'implementazione.

- ▶ Lato server, per i decision tree: la classe astratta `DecisionTreeFormat` dichiara gli attributi `extension` e `mime_type` e i metodi astratti `parse(text)` e `serialize(tree)`, realizzati da `JsonDecisionTreeFormat` e `CsvDecisionTreeFormat`. Le due implementazioni sono raccolte in un registro dal quale la strategia viene selezionata per nome, con `format_by_name(name)`, quando il formato è indicato nella richiesta di esportazione, oppure per estensione del file, con `format_for_filename(filename)`, quando è desunto dal file caricato in importazione. Le rotte si limitano a richiedere la strategia e a delegarle la conversione.
- **Conseguenze:** l'aggiunta di un formato richiede la sola introduzione di una nuova implementazione dell'interfaccia, senza modifiche ai moduli chiamanti né alle viste, e ciascuna strategia è verificabile con unit test indipendenti dalle altre. La presenza di due realizzazioni distinte, una per lato, riflette la ripartizione delle responsabilità secondo il possesso del dato: il dispositivo appartiene al client, che lo importa ed esporta senza coinvolgere il server, mentre il catalogo dei decision tree è posseduto dal backend, che ne governa quindi anche le conversioni.

3.6.4. Tabella riassuntiva

Pattern	Ambito	Moduli coinvolti
Adapter	Frontend	<code>ApiClientService/FetchApiClient</code> , <code>NotificationService/NotificationManager</code>
Repository	Backend	<code>IDecisionTreeRepository</code> , <code>JsonDecisionTreeRepository</code> , <code>DecisionTreeService</code>
Facade	Frontend, Backend	<code>importDeviceFromFile()</code> , <code>exportReportPdf()</code> , hook applicativi (<code>useSessionRunner</code> , <code>useResult</code>), <code>DecisionTreeService</code> , <code>create_device()</code> , <code>create_asset()</code>
Factory Method	Frontend, Backend	<code>create_app()</code> , <code>create_decision_tree_blueprint()</code> , <code>create_assets_blueprint()</code> , <code>normalize_tree()</code> , <code>_normalize_node()</code> , <code>createNode()</code> , metodi statici di costruzione delle entità di dominio, <code>from_string()</code> dei tipi enumerati
Observer	Frontend	<code>DeviceStore</code> , <code>SessionStore</code> , <code>TreeStore</code> e relativi hook selettori
Proxy	Frontend	componente <code>RequireSession</code> , <code>queryClient</code> su <code>DecisionTreeService</code>
Strategy	Frontend, Backend	<code>DeviceFileFormat</code> , <code>jsonDeviceFormat</code> , <code>csvDeviceFormat</code> , <code>formatForFile()</code> ; <code>DecisionTreeFormat</code> , <code>JsonDecisionTreeFormat</code> ,

		CsvDecisionTreeFormat, format_by_name(), format_for_filename()
--	--	---

4. Decomposizione del sistema

Questa sezione descrive l'organizzazione interna delle due unità del sistema. Per ciascun livello sono indicati i moduli che lo compongono, la responsabilità di ciascuno e le dipendenze verso gli altri livelli.

4.1. Diagramma dei package

4.2. Frontend

Il frontend è realizzato come **Single Page Application** in React. Le pagine non contengono logica di business: compongono l'interfaccia e delegano recupero dei dati, validazioni e transizioni di stato a hook applicativi dedicati, che a loro volta si appoggiano su store condivisi e su service.

4.2.1. Organizzazione a livelli

Il frontend è organizzato in cinque livelli:

Livello	Contenuto
Presentation	Pagine e componenti React, instradamento con React Router.
Application	Hook applicativi e service che orchestrano i casi d'uso.
State Management	Store Zustand distribuiti per area funzionale.
Infrastructure	Comunicazione HTTP, memorizzazione dei risultati, notifiche.
Domain	Entità di business, regole pure, schemi di validazione Zod.

La dipendenza fra livelli è a senso unico: Presentation dipende da Application; Application dipende da State, Domain e Infrastructure; Domain non dipende da alcun altro livello. È questo vincolo a rendere sostituibile un dettaglio tecnico senza toccare le pagine.

4.2.2. Presentation Layer

4.2.3. Presentation Layer

Il livello comprende otto pagine. Sette di esse corrispondono a una rotta dell'applicazione; ResultPage è invece resa da SessionRunnerPage al completamento della sessione.

- **HomePage**: creazione di un nuovo dispositivo, importazione da file o ripresa di una sessione salvata.
- **DeviceFormPage**: raccolta dei dati descrittivi del dispositivo, in creazione o in modifica.
- **DeviceAssetManagementPage**: gestione dell'elenco degli asset associati al dispositivo.
- **AssetFormPage**: creazione o modifica di un asset.
- **DeviceSummaryPage**: riepilogo di dispositivo e asset prima dell'avvio della valutazione.

- **SessionRunnerPage**: conduce l'intera valutazione guidata attraverso quattro fasi — riepilogo dell'avanzamento, selezione dell'asset, selezione del requisito con le relative dipendenze, esecuzione dell'albero decisionale — governate da `useSessionRunner`.
- **ResultPage**: consultazione degli esiti e loro esportazione.
- **DecisionTreeCatalogPage**: elenco dei decision tree disponibili, dettaglio, importazione, esportazione ed eliminazione.

I componenti condivisi principali comprendono:

- **Page**: layout comune delle pagine e gestione del titolo e della navigazione.
- **Loading**: indicatore dello stato di caricamento.
- **Field**: componente riutilizzabile per la presentazione dei campi dei form.
- **StatusBadge**: etichetta per la rappresentazione degli stati.
- **NoActiveSession**: messaggio e azione mostrati quando non esiste una sessione attiva.
- **Esito**: etichetta che rappresenta lo stato di valutazione.
- **GrafoDecisionTree**: rappresentazione grafica dell'albero, con evidenziazione del nodo corrente e del percorso già intrapreso.
- **RequireSession**: componente di guardia che avvolge le pagine richiedenti una sessione attiva e reindirizza alla pagina iniziale in sua assenza.

L'instradamento è definito in `App.tsx`, che associa un percorso a sette delle otto pagine e applica `RequireSession` alla rotta `/session`.

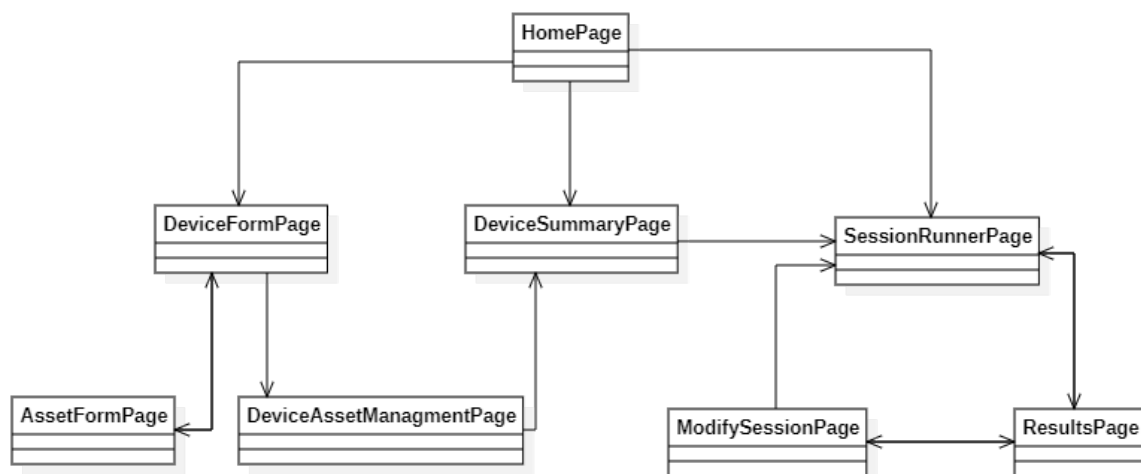


Figura 1: Diagramma delle pagine

4.2.4. Application Layer

La logica operativa risiede in due famiglie di moduli: gli **hook**, al servizio di una sola pagina, e i **service**, che raccolgono operazioni utili in più punti.

Gli hook espongono alla pagina il solo stato da mostrare e le azioni da collegare ai comandi, trattenendo la sequenza di passaggi che li produce.

- **useSessionRunner**: conduce la valutazione guidata, dalla selezione dell'asset e del requisito fino alla registrazione dell'esito; carica l'albero al momento opportuno e riporta il percorso al punto raggiunto quando una sessione viene ripresa.
- **useResult**: gestisce la consultazione degli esiti, consentendo di scendere dall'asset al singolo requisito e ricostruendo la sequenza di domande e risposte che ha condotto al risultato.
- **useSessionModify**: prepara la ripresa o la rivalutazione di un requisito e determina quali altri requisiti ne dipendono, poiché rifacendolo vanno rifatti anche quelli.

I service raccolgono le operazioni indipendenti dalla singola pagina.

- **DeviceService**: creazione manuale, importazione da file, creazione e modifica di un asset, esportazione del dispositivo. È realizzato come insieme di funzioni esportate anziché come classe.
- **DecisionTreeService**: recupero dell'elenco del catalogo e del singolo albero, importazione ed esportazione. È l'unico service realizzato come classe e riceve nel costruttore l'implementazione di `ApiClientService`, così da poter essere collaudato con un doppio di test.
- **SessionService**: produzione e riletture validate del file di sessione, e ri-esportazione delle regole di costruzione del piano di valutazione.
- **deviceFileFormats**: raccoglie i formati con cui il dispositivo può essere scritto e letto, oggi JSON e CSV, ciascuno con le proprie regole di serializzazione e interpretazione.

4.2.5. State Management

Lo stato condiviso fra più pagine è distribuito su tre store Zustand separati per area funzionale, collegati da una dipendenza controllata fra `DeviceStore` e `SessionStore`.

Store	Stato	Azioni
DeviceStore	Dispositivo in lavorazione e relativo payload.	setDevice, updateDeviceDetails, addAsset, updateAsset, removeAsset, reset
SessionStore	Sessione di valutazione: coppie asset-requisito, coppia corrente, percorsi ed esiti.	start, ensureSession, resume, syncProgress, completeCurrent, select, reset

TreeStore	Albero del requisito corrente, nodo attuale, cronologia delle risposte e cursore.	loadTree, hydrate, answer, goBack, goForward, reset
-----------	---	---

Ogni componente dichiara tramite un selettore la porzione di stato che gli occorre e viene aggiornato solo quando quella porzione cambia. Le modifiche passano esclusivamente da azioni dichiarate, mai da scritture dirette.

I tre store non sono del tutto indipendenti: quando il dispositivo viene sostituito o modificato, DeviceStore azzerava SessionStore. Si tratta di una dipendenza deliberata e a senso unico, che realizza un vincolo di dominio — una valutazione riferita a un dispositivo nel frattempo mutato produrrebbe esiti privi di significato.

Il cursore di TreeStore merita una precisazione: goBack e goForward spostano la posizione lungo la cronologia senza scartare le risposte successive, che vengono invalidate solo quando l'utente risponde diversamente a un nodo già risposto.

4.2.6. Infrastructure Layer

- **ApiClientService e FetchApiClient:** l'interfaccia con cui il resto del codice richiede dati al backend e la sua realizzazione concreta su fetch. Il metodo privato request() concentra la composizione dell'indirizzo, la serializzazione del corpo e la traduzione degli esiti: tanto un'assenza di rete quanto una risposta non riuscita diventano un ApiError, che espone il messaggio e, quando disponibile, il codice di stato.
- **queryClient:** istanza di TanStack Query impiegata in modo imperativo per conservare gli alberi già richiesti. La configurazione disabilita scadenza e ritentativi, coerentemente con la natura immutabile del dato durante una sessione.
- **ReportService e reportData:** il report di conformità viene costruito interamente nel client. reportData raccoglie dal dispositivo e dalla sessione gli esiti delle valutazioni, gli stati aggregati di asset e dispositivo e i percorsi logici seguiti. Quando necessario, recupera dal catalogo i decision tree associati ai requisiti per ricostruire le domande e le risposte del percorso.

ReportService utilizza questi dati per generare il documento PDF tramite la libreria @react-pdf/renderer. La struttura grafica del documento è definita dal componente ReportDocument, mentre il file prodotto viene reso disponibile per il download dell'utente. Questa soluzione evita di trasferire la sessione al backend, che non la possiede né la conserva.

- **NotificationService e NotificationManager:** l'interfaccia per avvisare l'utente e la sua realizzazione su react-hot-toast.

Grazie a questa separazione, sostituire il modo in cui si dialoga con il backend o si presentano gli avvisi non tocca né le pagine né i service.

4.2.7. Domain Layer

Il livello rappresenta il problema applicativo in modo indipendente da React, dal routing e dal backend. Comprende le entità di dominio e tre insiemi di regole scritte come funzioni pure, e quindi verificabili in isolamento.

- **treeRules:** percorre l'albero. `resolveNodeId()` ricava il nodo corrente ripercorrendo le risposte dalla radice, `nodeById()` reperisce un nodo per identificatore, `currentOutcome()` riconosce il raggiungimento di una foglia restituendone l'esito, `describePath()` ricostruisce la sequenza di domande e risposte da presentare nei risultati.
- **sessionRules:** governa la valutazione. `getEvaluationProgress()` calcola l'avanzamento; `getEvaluationStatus()`, `getRequirementStatus()`, `getAssetStatus()` e `getDeviceStatus()` riassumono più esiti in uno solo secondo una priorità fissa che fa prevalere il fallimento e la valutazione in corso sugli esiti positivi.
- **treeLayout:** dispone i nodi in colonne e livelli per il disegno del grafo, producendo coordinate astratte così da restare indipendente dalla tecnologia di rendering.

La validazione dei dati in ingresso (input dell'utente, risposte del backend, contenuto dei file importati) è affidata a schemi **Zod** definiti accanto alle entità.

4.2.8. Flussi applicativi

Il flusso dei dati è unidirezionale. Le pagine non modificano lo stato condiviso né dialogano con il backend: raccolgono l'azione dell'utente e la inoltrano all'hook, che decide come procedere. Se occorre un dato che il client non possiede, l'hook lo richiede al service, che passa dall'Infrastructure Layer; altrimenti agisce direttamente sugli store.

I flussi principali sono i seguenti.

- **Nuova valutazione:** da `HomePage` si passa a `DeviceFormPage` per i dati del dispositivo, quindi a `DeviceAssetManagementPage` e `AssetFormPage` per gli asset, infine a `DeviceSummaryPage` per il riepilogo. Il dispositivo resta nel `DeviceStore` per tutto il percorso.
- **Importazione di un dispositivo:** il formato è riconosciuto dall'estensione tramite `formatForFile()`, il contenuto è letto e validato con `Zod`, quindi i metadati del dispositivo e ciascun asset sono inviati al backend, che assegna gli identificativi mancanti e deriva i requisiti applicabili.
- **Avvio della valutazione:** da `DeviceSummaryPage` si entra in `SessionRunnerPage`; la sessione è creata a partire dal piano prodotto da `buildPlan()`. Se una sessione compatibile è già in corso, `ensureSession()` la riprende anziché ricominciare.
- **Esecuzione del decision tree:** scelto il requisito, l'albero è caricato e il percorso riportato al punto raggiunto tramite `hydrate()`. Ogni risposta aggiorna il percorso nel client e allinea la sessione, cosicché un salvataggio colga sempre lo stato corrente. Raggiunta una foglia, l'esito è registrato con `completeCurrent()`.

- **Ripetizione di una valutazione:** dalla vista dell'asset è possibile selezionare nuovamente un requisito già valutato; `select()` riporta la sessione in corso e la valutazione riparte dalla radice dell'albero.
- **Consultazione degli esiti:** valutate tutte le coppie, `ResultPage` consente di scendere dall'asset al requisito e rivedere domande e risposte.
- **Salvataggio e ripresa da file:** la sessione può essere scaricata in qualsiasi momento e ricaricata da `HomePage`, previa validazione dello schema.
- **Esportazione:** il dispositivo è scritto in JSON o CSV dal client; il decision tree è prodotto dal backend, che risponde all'endpoint di esportazione nel formato richiesto.

4.2.9. Caratteristiche di qualità

4.2.9.1. Manutenibilità

La separazione in livelli, i selettori a granularità fine e la centralizzazione delle operazioni nei service circoscrivono l'impatto delle modifiche al livello competente. La proprietà è verificabile nella storia del prodotto: la rappresentazione grafica del decision tree, inizialmente scritta come grafica SVG, è stata sostituita da `React Flow` senza che la regola `treeLayout` del `Domain Layer` subisse alcuna modifica.

4.2.9.2. Scalabilità

La struttura attuale sostiene estensioni progressive:

- nuove pagine senza impatto sul dominio, essendo la logica concentrata in hook e service;
- nuovi flussi applicativi mediante hook dedicati;
- nuovi formati di scambio mediante una nuova realizzazione di `DeviceFileFormat`;
- sostituzione delle realizzazioni infrastrutturali con impatto circoscritto.

Tali proprietà discendono da una disciplina di codifica e non da un vincolo imposto dagli strumenti.

4.2.10. Elementi principali

4.2.10.1. Servizi applicativi

I servizi raccolgono le operazioni applicative indipendenti dalla singola vista. Con l'eccezione di `DecisionTreeService`, sono realizzati come insiemi di funzioni esportate, non avendo stato da conservare fra le invocazioni.

4.2.10.1.1. DecisionTreeService

Servizio di accesso al catalogo degli alberi decisionali. Unica classe fra i servizi, in quanto conserva il riferimento al client HTTP ricevuto in costruzione.

Attributi

- `api: ApiClientService`: client HTTP, ricevuto nel costruttore con `FetchApiClient` come valore predefinito, per consentirne la sostituzione nei test.

Metodi

- `listTrees(): Promise<DecisionTreeSummary[]>`: restituisce l'elenco del catalogo, validato e conservato in cache;
- `getTree(requirementId): Promise<DecisionTree>`: carica un singolo albero e ne costruisce l'entità;
- `deleteTree(requirementId): Promise<void>`: elimina un albero e invalida la cache;
- `importTree(file): Promise<DecisionTree>`: trasmette il file come multipart/form-data e invalida l'elenco memorizzato;
- `exportTree(requirementId, format): Promise<void>`: richiede l'albero nel formato indicato e ne avvia il download.

Il modulo esporta inoltre l'istanza condivisa `decisionTreeService`.

4.2.10.1.2. DeviceService

Modulo per la creazione, l'importazione e l'esportazione del dispositivo e dei suoi asset.

Attributi

- Nessuno: il modulo è realizzato come insieme di funzioni prive di stato.

Metodi

- `importDeviceFromFile(file): Promise<DeviceSaveResult>`: riconosce il formato, interpreta il contenuto e trasmette al backend dispositivo e asset, questi ultimi in parallelo;
- `exportDevice(device, format): void`: serializza il dispositivo e ne avvia il download;
- `createDeviceManually(payload): Promise<DeviceSaveResult>`: valida i dati e li trasmette al backend, che risolve l'identificatore;
- `createAsset(payload): Promise<Asset>`: valida i dati e costruisce l'asset dalla risposta, che ne include i requisiti applicabili;
- `updateAsset(existingAsset, payload): Promise<Asset>`: aggiorna l'asset localmente se la tipologia non è mutata, interroga altrimenti il backend, poiché i requisiti applicabili dipendono dalla tipologia.

4.2.10.1.3. SessionService

Modulo per la produzione e la riletture del file di sessione.

Attributi

- Nessuno: il modulo è realizzato come insieme di funzioni prive di stato.

Metodi

- `toSessionFile(session)`: Session: aggiorna l'istante di salvataggio;
- `parseSessionFile(text)`: Session: interpreta e valida il contenuto, distinguendo il file non interpretabile da quello non conforme;
- `downloadSession(session)`: void: serializza la sessione e ne avvia il download;
- `loadSessionFromJson(file)`: Promise<Session>: verifica il formato, legge e valida.

4.2.10.1.4. ReportService

Modulo per la produzione del report di conformità finale.

Attributi

- Nessuno: il modulo espone una sola funzione priva di stato.

Metodi

- `exportReportPdf(session)`: Promise<void>: costruisce i dati del report, li rende in un documento PDF tramite ReportDocument e ne avvia il download.

4.2.10.1.5. reportData

Modulo che raccoglie i dati del report a partire dalla sessione, separando la costruzione del contenuto dalla resa grafica. Definisce le strutture ReportData, ReportAssetEntry, ReportRequirementEntry e ReportRequirementSummary.

Attributi

- Nessuno: il modulo espone una sola funzione priva di stato.

Metodi

- `buildReportData(session)`: Promise<ReportData>: risolve le denominazioni dei requisiti dagli alberi, calcola gli stati aggregati tramite sessionRules e ricostruisce il percorso di ciascuna valutazione con describePath(). L'assenza di un albero dal catalogo non impedisce la generazione: il percorso è omissso e pathAvailable lo segnala.

4.2.10.2. Infrastruttura

Il livello infrastrutturale raccoglie i moduli che mediano fra la logica applicativa e i meccanismi tecnici esterni: comunicazione HTTP, conservazione dei risultati e notifiche.

4.2.10.2.1. ApiClientService

Interfaccia che dichiara le operazioni HTTP di cui la logica applicativa ha bisogno, espresse in termini tipizzati e indipendenti dal meccanismo sottostante.

Metodi

- `get<T>(path): Promise<T>;`
- `post<T>(path, body): Promise<T>;`
- `postFormData<T>(path, body): Promise<T>;`
- `delete<T>(path): Promise<T>.`

L'interfaccia dichiara le sole operazioni effettivamente impiegate dai chiamanti.

4.2.10.2.2. FetchApiClient

Realizzazione concreta di `ApiClientService` fondata sulla funzione `fetch` del browser.

Attributi

- `baseUrl: string`: prefisso degli indirizzi, vuoto per impostazione predefinita; il client compone così percorsi relativi, che raggiungono il backend tramite reverse proxy.

Metodi

- `buildUrl(path): string (privato)`: compone l'indirizzo completo;
- `request<T>(path, init): Promise<T> (privato)`: esegue la richiesta e ne traduce l'esito. Tanto un'assenza di rete quanto una risposta non riuscita divengono un `ApiError`; un corpo vuoto è restituito come valore indefinito;
- `get, post, postFormData, delete`: realizzano l'interfaccia delegando a `request()`.

4.2.10.2.3. ApiError

Errore applicativo prodotto in luogo delle eccezioni tecniche della comunicazione HTTP.

Attributi

- `status: number | null`: codice di stato HTTP, assente quando l'errore è di rete.

Estendendo `Error`, conserva messaggio e traccia di esecuzione. La presenza o assenza di `status` consente ai chiamanti di distinguere il fallimento della richiesta dall'irraggiungibilità del server.

4.2.10.2.4. NotificationService

Interfaccia per la segnalazione di esiti all'utente, indipendente dalla libreria impiegata.

Metodi

- `success(message): void`;
- `error(message): void`;
- `errorWithFallback(message): void`: mostra un messaggio predefinito quando quello ricevuto è vuoto, evitando notifiche prive di contenuto.

4.2.10.2.5. NotificationManager

Realizzazione di NotificationService fondata su *react-hot-toast*, alla quale delega coda, temporizzatore di scomparsa e impilamento dei messaggi.

Attributi

- Nessuno: la classe è priva di stato proprio.

4.2.10.2.6. queryClient

Istanza di QueryClient impiegata per conservare i risultati già ottenuti dal catalogo.

Configurazione

- staleTime e gcTime infiniti: il dato è immutabile per la durata della sessione;
- retry disabilitato: la conservazione serve a non ripetere richieste, non a compensare l'inaffidabilità della comunicazione.

È impiegata in modo imperativo tramite `fetchQuery()`, senza montare alcun *provider*.

4.2.10.3. Formati di file

4.2.10.3.1. deviceFileFormats

Modulo che raccoglie i formati con cui il dispositivo può essere scritto e letto, dietro un'interfaccia comune.

Interfaccia DeviceFileFormat

- extension: string: estensione del file;
- mimeType: string: tipo di contenuto;
- serialize(device): string: produce la rappresentazione testuale del dispositivo;
- parse(text): DeviceImport: interpreta il contenuto e lo valida.

Realizzazioni

- jsonDeviceFormat: formato JSON, con asset annidati nel dispositivo;
- csvDeviceFormat: formato tabellare, con intestazione attesa, codifica dei campi contenenti separatori e asset rappresentati su righe successive.

Funzioni

- formatForFile(file): DeviceFileFormat | null: seleziona il formato in base all'estensione, restituendo un valore nullo se non riconosciuta.

L'aggiunta di un formato richiede la sola introduzione di una nuova realizzazione dell'interfaccia, senza modifiche ai moduli chiamanti.

4.2.10.4. Store

Lo stato condiviso fra più viste è distribuito su tre store Zustand indipendenti, ciascuno dedicato a un'area funzionale. I componenti vi accedono tramite hook selettori, che circoscrivono la sottoscrizione alla sola porzione di stato utilizzata.

4.2.10.4.1. DeviceStore

Store del dispositivo in lavorazione e dei suoi asset.

Attributi

- `device`: `Device` | `null`: dispositivo corrente;
- `payload`: `unknown`: dati grezzi da cui il dispositivo deriva, conservati per la successiva esportazione.

Metodi

- `setDevice(device, payload)`: sostituisce il dispositivo corrente;
- `updateDeviceDetails(patch)`: aggiorna i dati descrittivi;
- `addAsset(asset)`, `updateAsset(asset)`, `removeAsset(assetId)`: modificano l'elenco degli asset;
- `reset()`: ripristina lo stato iniziale.

La sostituzione del dispositivo e la modifica dei suoi dati descrittivi azzerano `SessionStore`. Le operazioni sugli asset non lo fanno: la sessione incorpora il proprio dispositivo e non legge da `DeviceStore` mentre è in corso. La coerenza è verificata al successivo avvio della valutazione, dove `ensureSession()` confronta i piani e riprende la sessione se il piano è invariato, preservandone l'avanzamento.

4.2.10.4.2. SessionStore

Store della sessione di valutazione.

Attributi

- `session`: `Session` | `null`: sessione corrente, comprensiva di dispositivo, valutazioni e coppia asset-requisito in corso.

Metodi

- `start(device)`: avvia una nuova sessione con identificatore generato;
- `ensureSession(device)`: riprende la sessione esistente se riferita al medesimo dispositivo e coerente con il piano attuale, avviandone altrimenti una nuova;
- `resume(session)`: adotta una sessione ricevuta dall'esterno, tipicamente da file;

- `syncProgress(nodeId, path)`: registra nodo corrente e percorso parziale senza chiudere la valutazione;
- `completeCurrent(outcome, path)`: registra l'esito raggiunto;
- `select(assetId, requirementId)`: attiva una coppia asset-requisito;
- `reset()`: ripristina lo stato iniziale.

4.2.10.4.3. TreeStore

Store dell'albero decisionale in esecuzione e della navigazione al suo interno.

Attributi

- `tree`: `DecisionTree` | `null`: albero del requisito corrente;
- `currentNodeId`: `string` | `null`: nodo raggiunto;
- `history`: `PathStep[]`: cronologia delle risposte fornite;
- `cursor`: `number`: posizione corrente lungo la cronologia.

Metodi

- `loadTree(tree)`: carica un albero posizionandosi sulla radice;
- `hydrate(tree, steps)`: carica un albero e ne ricostruisce la posizione applicando le risposte già registrate;
- `answer(value)`: registra una risposta; se coincide con quella già presente in quella posizione avanza soltanto il cursore, conservando le risposte successive, altrimenti le scarta;
- `goBack()`, `goForward()`: spostano il cursore lungo la cronologia senza alterarla;
- `reset()`: ripristina lo stato iniziale.

La distinzione fra cronologia e cursore consente all'utente di rivedere le risposte già date senza perderle: sono invalidate soltanto quando risponde diversamente a un nodo già attraversato.

4.2.10.5. Hook applicativi

Gli hook costituiscono il livello di orchestrazione fra vista, stato globale e servizi.

Incapsulano i flussi operativi e gli effetti collaterali, esponendo alla pagina il solo stato da mostrare e le azioni da collegare ai comandi.

4.2.10.5.1. useSessionRunner

Hook orchestratore della valutazione guidata. Governa l'avanzamento fra le quattro fasi dell'interfaccia — riepilogo, asset, dettaglio del requisito ed esecuzione dell'albero — e coordina il caricamento dell'albero con la registrazione degli esiti.

Parametri

- Nessuno.

Dipendenze

- SessionStore: lettura della sessione e azioni syncProgress, completeCurrent, select, reset;
- TreeStore: navigazione nell'albero tramite hydrate, answer, goBack, goForward, reset;
- DecisionTreeService: caricamento degli alberi;
- SessionService: salvataggio della sessione su file;
- sessionRules, treeRules: calcolo dell'avanzamento e dell'esito corrente.

Stato interno

- phase: fase dell'interfaccia; è inizializzata rientrando direttamente nell'esecuzione dell'albero quando si riprende una sessione interrotta a metà di un requisito;
- status: stato del caricamento dell'albero;
- selectedAssetId, selectedRequirementId: selezione in corso di navigazione;
- requirementDetail: denominazione e dipendenze del requisito selezionato.

Comportamento

- Entrando nella fase di esecuzione carica l'albero della coppia corrente e ne ricostruisce la posizione con hydrate(), applicando le risposte già registrate;
- a ogni variazione del nodo corrente allinea la sessione con syncProgress(), cosicché un salvataggio colga sempre lo stato aggiornato;
- nel dettaglio del requisito carica l'albero per ricavarne denominazione e dipendenze, senza alterare TreeStore;
- all'avvio di un requisito azzera l'albero precedente, poiché due coppie riferite allo stesso codice condividerebbero l'identificatore e l'albero residuo contaminerebbe la nuova valutazione;
- raggiunta una foglia, registra l'esito e riporta alla vista dell'asset.

Valori esposti

- Contesto: session, isCompleted, progress, asset, requirementId, selectedAsset, requirementDetail;
- Albero: tree, currentNode, currentNodeId, path, outcome;

- Navigazione: `answer`, `goBack`, `goForward`, `canGoBack`, `canGoForward`;
- Azioni: `openAsset`, `openRequirement`, `startRequirement`, `backToDashboard`, `backToAsset`, `confirmOutcome`, `saveSession`, `endSession`.

Le proprietà `canGoBack` e `canGoForward` derivano dalla posizione del cursore: l'avanzamento è consentito soltanto sui nodi già risposti.

4.2.10.5.2. `useResult`

Hook per la consultazione degli esiti, che governa la discesa dall'asset al singolo requisito e la ricostruzione del percorso logico seguito.

Parametri

- Nessuno.

Dipendenze

- `SessionStore`: lettura della sessione;
- `DecisionTreeService`: caricamento dell'albero del requisito selezionato;
- `treeRules`: ricostruzione del percorso tramite `describePath()`.

Stato interno

- `selectedAssetId`, `selectedRequirementId`: selezione corrente;
- `pathQuestions`: sequenza di domande e risposte del requisito selezionato, nulla finché il caricamento è in corso.

Comportamento

- Alla selezione di un requisito carica l'albero corrispondente e ne ricostruisce il percorso a partire dalle risposte registrate nella sessione;
- se l'albero non è più disponibile nel catalogo restituisce un percorso vuoto anziché interrompere la consultazione;
- la selezione di un asset azzerà quella del requisito.

Valori esposti

- `session`, `selectedAssetId`, `selectedRequirementId`, `pathQuestions`;
- `selectAsset`, `selectRequirement`, `clearAsset`, `clearRequirement`.

4.3. Backend

Il backend è un'applicazione Flask esposta come singola API REST stateless, organizzata in quattro livelli.

4.3.1. Organizzazione a livelli

Livello	Contenuto
Presentation	Blueprint Flask che espongono gli endpoint REST.
Application	Service che orchestrano i casi d'uso.
Domain	Entità e loro regole, senza dipendenze da Flask o dal filesystem.
Persistence	Interfaccia e implementazione di accesso al catalogo.

Presentation dipende da Application, Application da Domain e Persistence, Domain da nulla. La direzione è verificata dai test di dominio, che non richiedono alcun contesto applicativo Flask.

4.3.2. Presentation Layer

Il livello raggruppa quattro blueprint. Ogni rotta riceve la richiesta, delega l'elaborazione al service competente e traduce l'esito in una risposta HTTP con il codice di stato appropriato, senza contenere logica di business.

L'applicazione è composta da `create_app()`, che costruisce il repository, lo inietta nel `DecisionTreeService` e passa quest'ultimo alle factory dei blueprint che ne hanno bisogno. Il blueprint dei dispositivi, le cui rotte invocano direttamente la funzione di validazione e non dipendono da collaboratori costruiti a runtime, è registrato senza factory.

4.3.3. Application Layer

- **`create_device()`**: valida i metadati del dispositivo (nome, sistema operativo, descrizione) e ne costruisce l'entità, rispettando l'id se fornito o generandone uno con `uuid4`. È il punto unico di validazione, condiviso dalla creazione manuale e dall'importazione.
- **`create_asset()`**: valida i campi dell'asset e, quando `requirements` non è fornito, deriva i requisiti applicabili interrogando `DecisionTreeService.list_requirement_ids_for_type()`.
- **`DecisionTreeService`**: unica classe del livello. Carica un albero tramite il repository, ne verifica l'integrità e lo normalizza; espone inoltre `list_trees()`, `list_requirement_ids_for_type()` e `import_tree()` e `delete_tree()`.

Le prime due sono realizzate come funzioni esportate anziché come classi, non avendo stato né collaboratori da conservare fra le invocazioni.

4.3.4. Domain Layer

Il livello comprende le entità `Device`, `Asset`, `DecisionTree` e `Session`, realizzate come dataclass immutabili, oltre alla classe astratta `Node` e alle dataclass concrete `QuestionNode` e `LeafNode`. Le entità non dipendono da librerie web né dall'accesso diretto ai file. Le operazioni di navigazione sono definite come metodi delle entità: `DecisionTree.get_node()` reperisce un nodo per identificatore, `Node.next()` restituisce il successore per il ramo scelto, `Node.verdict()` l'esito di una foglia.

Session e le strutture correlate (PathStep, Evaluation, Current) sono definite lato server per rappresentare il file di sessione, ma non sono esposte da alcuna rotta: la sessione è gestita interamente dal client.

4.3.5. Persistence Layer

Il livello isola l'accesso ai dati dietro l'interfaccia IDecisionTreeRepository, realizzata da JsonDecisionTreeRepository sui file in backend/data/decision_trees/.

4.3.6. Endpoint REST

Metodo	Percorso	Descrizione
GET	/health	Verifica di disponibilità del servizio.
POST	/devices	Valida i metadati del dispositivo, risolve l'id e restituisce l'entità serializzata con assets sempre vuoto. Nessuna scrittura su disco. Contratto condiviso da creazione manuale e importazione.
POST	/assets	Valida i campi dell'asset e ne deriva i requisiti applicabili quando non forniti, interrogando il catalogo. Risolve l'id. Nessuna scrittura su disco.
GET	/decision-trees	Restituisce l'elenco sintetico dei decision tree disponibili.
GET	/decision-trees/{requirementId}	Carica il decision tree indicato, ne verifica l'integrità e lo restituisce normalizzato. Risponde 404 se assente.
POST	/decision-trees/import	Riceve un file JSON o CSV come multipart/form-data, ne valida la struttura e lo registra nel catalogo tramite il repository. Risponde 400 in caso di struttura non valida.
DELETE	/decision-trees/{requirementId}	Elimina definitivamente dal catalogo il decision tree indicato. Risponde 404 se l'albero non esiste e 204 in caso di eliminazione riuscita.
GET	/decision-trees/{requirementId}/export	Restituisce il decision tree nel formato indicato dal parametro format, json o csv.

4.3.7. Elementi principali

4.3.7.1. Presentation Layer

4.3.7.1.1. create_app

Funzione factory che costruisce e configura l'istanza di Flask.

Comportamento

- Registra i blueprint privi di collaboratori (health_bp, devices_bp);

- costruisce `JsonDecisionTreeRepository` sulla cartella dei dati e lo inietta in `DecisionTreeService`;
- costruisce i blueprint che dipendono dal servizio tramite le rispettive factory;
- abilita le intestazioni CORS.

È l'unico punto del backend in cui compaiono i nomi delle classi concrete: la direzione delle dipendenze è pertanto verificabile ispezionando un solo file. La costruzione ripetibile consente inoltre a ciascun test di disporre di un'istanza isolata.

4.3.7.1.2. **health_bp**

Blueprint che espone GET `/health` per la verifica di disponibilità del servizio. Non dipende da alcun collaboratore ed è registrato direttamente.

4.3.7.1.3. **devices_bp**

Blueprint che espone POST `/devices`. Delega la validazione a `create_device()`, traduce `InvalidDeviceDataError` in una risposta 400 e serializza l'entità con `_serialize_device()`, che restituisce sempre assets vuoto: gli asset sono trasmessi separatamente.

4.3.7.1.4. **create_assets_blueprint**

Factory del blueprint che espone POST `/assets`. Riceve `DecisionTreeService` e lo trasmette a `create_asset()`, necessario per derivare i requisiti applicabili. Traduce `InvalidAssetDataError` in una risposta 400.

4.3.7.1.5. **create_decision_tree_blueprint**

Factory del blueprint del catalogo, che espone le cinque rotte dei decision tree: elenco, importazione, lettura, eliminazione ed esportazione. Traduce `DecisionTreeNotFoundError` in 404 e `InvalidDecisionTreeError` in 400; l'esportazione seleziona il formato tramite `format_by_name()` e restituisce 400 se non riconosciuto.

4.3.7.2. **Application Layer**

4.3.7.2.1. **device_service**

Modulo che espone `create_device(data) -> Device`, punto unico di costruzione validata del dispositivo, condiviso da creazione manuale e importazione. È realizzato come funzione, non avendo stato né collaboratori da conservare.

4.3.7.2.2. **asset_service**

Modulo che espone `create_asset(data, decision_tree_service) -> Asset`. Quando il campo `requirements` non è fornito, interroga `list_requirement_ids_for_type()` per derivare i requisiti applicabili alla tipologia dell'asset, quindi delega la validazione ad `Asset.create()`.

4.3.7.2.3. **DecisionTreeService**

Unica classe del livello, in quanto conserva il riferimento al repository ricevuto in costruzione.

Attributi

- `_repository`: `IDecisionTreeRepository`: accesso al catalogo, iniettato dal costruttore.

Metodi

- `get_tree(requirement_id)` -> `DecisionTree`: carica l'albero e lo normalizza, sollevando `DecisionTreeNotFoundError` se assente;
- `delete_tree(requirement_id)` -> `None`: elimina un albero dal catalogo;
- `list_trees()` -> `list[dict]`: restituisce l'elenco sintetico del catalogo;
- `list_requirement_ids_for_type(asset_type)` -> `list[str]`: seleziona i requisiti applicabili a una tipologia di asset in base al campo `appliesTo`;
- `import_tree(content, filename)` -> `tuple[DecisionTree, str]`: riconosce il formato dall'estensione, ne interpreta il contenuto, lo valida e lo registra tramite il repository.

Il modulo espone inoltre le funzioni `normalize_tree()` e `_normalize_node()`, che costruiscono le entità di dominio a partire dai dati grezzi scegliendo la classe concreta in base al campo `type` del nodo.

4.3.7.2.4. `decision_tree_format`

Modulo che raccoglie i formati di scambio dei decision tree dietro un'interfaccia comune.

Interfaccia `DecisionTreeFormat` (classe astratta)

- `extension, mime_type`: estensione e tipo di contenuto;
- `parse(text)` -> `dict`: interpreta il contenuto testuale;
- `serialize(tree)` -> `str`: produce la rappresentazione del formato.

Realizzazioni

- `JsonDecisionTreeFormat`;
- `CsvDecisionTreeFormat`, che interpreta l'intestazione attesa e ricostruisce i nodi dalle righe successive.

Funzioni

- `format_by_name(name)`: seleziona il formato dal parametro della richiesta di esportazione;
- `format_for_filename(filename)`: seleziona il formato dall'estensione del file importato;
- `tree_to_dict(tree)`: produce la rappresentazione serializzabile dell'albero, comprensiva dell'involucro `schemaVersion` e `kind`.

Entrambe le funzioni di selezione restituiscono un valore nullo se il formato non è riconosciuto. L'aggiunta di un formato richiede la sola introduzione di una nuova realizzazione dell'interfaccia.

4.3.7.3. Persistence Layer

4.3.7.3.1. IDecisionTreeRepository

Classe astratta che dichiara le operazioni sul catalogo come se esso fosse disponibile in memoria.

Metodi

- `get(id)`: restituisce l'albero grezzo, o un valore nullo se assente;
- `save(decision_tree)`: registra un albero nel catalogo;
- `delete(id)`: elimina un albero;
- `list()`: restituisce gli identificatori disponibili.

4.3.7.3.2. JsonDecisionTreeRepository

Realizzazione dell'interfaccia su file JSON.

Attributi

- `_data_dir`: Path: cartella del catalogo.

Metodi

- `_path(id) -> Path` (privato): compone il percorso del file;
- `get, save, delete, list`: realizzano l'interfaccia operando sul filesystem. La cartella è creata al bisogno in fase di salvataggio e l'eliminazione di un file assente non produce errore.

I test dei servizi sostituiscono questa classe con `FakeDecisionTreeRepository`, che realizza la medesima interfaccia mantenendo gli alberi in memoria.

4.3.7.4. Eccezioni di dominio

Le condizioni di errore sono rappresentate da eccezioni dedicate, che il Presentation Layer traduce nel codice di stato appropriato.

- `InvalidDeviceDataError`: dati del dispositivo non conformi; risposta 400.
- `InvalidAssetDataError`: dati dell'asset non conformi; risposta 400.
- `InvalidDecisionTreeError`: albero strutturalmente non valido o formato non supportato; risposta 400.
- `DecisionTreeNotFoundError`: albero non presente nel catalogo; risposta 404.

L'impiego di eccezioni distinte consente alle rotte di determinare la risposta senza ispezionare il messaggio d'errore.

4.3.8. Testing

I test del backend risiedono in una cartella `tests/` separata, a specchio di `src/`, secondo la convenzione idiomatica di Pytest; il frontend adotta invece la convenzione idiomatica di Vitest, con i test collocati accanto al file verificato. Il Domain Layer è verificato con unit test; i service con test che sostituiscono il repository con `FakeDecisionTreeRepository`; le rotte con test di integrazione tramite il client di test di Flask. La qualità del codice è verificata in integrazione continua tramite `ruff` lato backend ed `eslint` lato frontend.

5. Diagrammi delle classi

5.1. Criteri di rappresentazione

Questa sezione descrive la struttura statica del dominio applicativo, cioè le entità che rappresentano i concetti di business su cui si fonda la valutazione di conformità EN 18031 e le relazioni che le legano.

Il dominio esiste in due realizzazioni distinte, una per ciascuna unità del sistema.

Nelle schede che seguono sono riportati, per ciascuna entità, il ruolo, gli attributi con il rispettivo tipo, i metodi che portano comportamento e la responsabilità complessiva. I metodi di accesso ai singoli campi sono riportati per completezza: nel frontend sono `getter`, nel backend proprietà di sola lettura.

5.2. Dominio frontend

Le entità del dominio frontend sono progettate secondo un modello immutabile a livello dell'oggetto: le proprietà sono private e le operazioni di modifica restituiscono nuove istanze invece di modificare direttamente i campi. Le collezioni contenute nelle entità sono gestite secondo questa convenzione dagli store e dai metodi `with*`. L'immutabilità garantisce che nessun modulo possa alterare uno stato condiviso in modo non tracciabile e si combina con la propagazione delle modifiche degli store, che rilevano il cambiamento per identità del riferimento.

Ciascuna entità è affiancata da uno o più schemi **Zod** che ne descrivono la forma attesa e dai quali sono derivati i tipi TypeScript corrispondenti. Un metodo statico di classe costituisce l'unico punto di costruzione a partire da dati esterni: valida il dato grezzo contro lo schema e costruisce l'istanza solo se la validazione ha successo. Si chiama **`create()`** in Device, Asset e DecisionTree, e **`parse()`** in Session, dove il dato di partenza è un file di sessione anziché una risposta del backend.

5.2.1. Device

Ruolo: rappresenta il dispositivo sottoposto a valutazione e aggrega gli asset che lo compongono.

Attributi

- `id: string` : identificatore univoco, assegnato dal backend o presente nel file importato.
- `name: string` : denominazione del dispositivo.
- `operatingSystem: string` : sistema operativo del dispositivo.
- `description: string` : descrizione testuale.
- `assets: Asset[]` : gli asset associati, in composizione.

Metodi

- `Get: id(): string, name(): string, operatingSystem(): string, description(): string, assets(): Asset[]` : restituiscono il corrispondente attributo.
- `withDetails(patch): Device` : restituisce un dispositivo con i dati descrittivi aggiornati.
- `withAssets/assets): Device, withAssetAdded(asset): Device, withAssetUpdated(asset): Device, withAssetRemoved(assetId): Device` : restituiscono un dispositivo con l'elenco degli asset modificato.
- `buildPlan(): { assetId, requirementId }[]` : costruisce l'elenco delle coppie asset-requisito da valutare, percorrendo gli asset e i requisiti assegnati a ciascuno.
- `toJSON(): object` : produce la rappresentazione serializzabile, usata dall'esportazione e dal file di sessione.
- `create(raw): Device` : costruisce l'entità validando il dato grezzo con `DeviceSchema`.

Responsabilità: raccogliere i dati descrittivi del dispositivo, custodire l'elenco degli asset e derivare il piano di valutazione che costituisce il punto di partenza di ogni sessione.

Schemi associati: `DeviceSchema` (forma completa), `DeviceCreateSchema` (dati raccolti nel form di creazione), `DeviceImportSchema` (forma attesa in un file importato, con `id` facoltativo e asset annidati).

5.2.2. Asset

Ruolo: rappresenta un elemento del dispositivo soggetto a valutazione, al quale sono associati i requisiti EN 18031 applicabili.

Attributi

- `id: string` : identificatore univoco all'interno del dispositivo.
- `name: string` : denominazione dell'asset.
- `type: AssetType` : categoria funzionale, ristretta ai valori `network`, `security`, `privacy` e `financial`.
- `description: string` : descrizione testuale.

- `sensitive: boolean` : indica se l'asset tratta dati sensibili.
- `requirements?: string[]` : codici dei requisiti assegnati, derivati dal backend in base al tipo.

Metodi

- `Get: id(): string, name(): string, type(): AssetType, description(): string, sensitive(): boolean, requirements(): string[] | undefined` : restituiscono il corrispondente attributo.
- `withDetails(patch): Asset` : restituisce un asset con i dati descrittivi aggiornati, conservando i requisiti assegnati.
- `toJSON(): object` : produce la rappresentazione serializzabile.
- `create(raw): Asset` : costruisce l'entità validando il dato grezzo con `AssetSchema`.

Responsabilità: custodire i metadati dell'asset e i requisiti a esso applicabili.

L'asset non conserva il proprio stato di valutazione: quello stato appartiene alla sessione ed è calcolato da `getAssetStatus()` a partire dalle valutazioni registrate. La scelta evita che la stessa informazione esista in due luoghi con il rischio di divergere.

5.2.3. DecisionTree

Ruolo: rappresenta l'albero decisionale che guida la valutazione di un requisito.

Attributi

- `requirementId: string` : codice del requisito, conforme all'espressione regolare che ne descrive il formato.
- `requirementName: string` : denominazione estesa del requisito.
- `version?: string` : versione dell'albero, nel formato a tre numeri.
- `appliesTo?: string[]` : tipi di asset ai quali il requisito si applica; è il campo su cui il backend deriva i requisiti applicabili.
- `dependencies?: string[]` : requisiti dai quali questo dipende.
- `rootNode: string` : identificatore del nodo radice.
- `nodes: Node[]` : collezione dei nodi in forma piatta.
- `message?: string` : messaggio restituito dal backend a seguito di un'importazione.

Metodi

- `Get: requirementId(): string, requirementName(): string, version(): string | undefined, appliesTo(): string[] | undefined, dependencies(): string[] |`

undefined, rootNode(): string, nodes(): Node[], message(): string | undefined : restituiscono il corrispondente attributo.

- getNode(id): Node : restituisce il nodo corrispondente all'identificatore, sollevando un errore se assente.
- toJSON(): object, create(raw): DecisionTree : come per le altre entità.

Responsabilità: rappresentare la struttura dell'albero e consentirne l'accesso per identificatore, operazione richiesta a ogni passo della navigazione.

I nodi sono conservati in una **lista piatta** e i collegamenti sono espressi per identificatore, anziché in una struttura di contenimento ricorsiva. La ragione decisiva è che rami distinti devono poter convergere sullo stesso nodo, situazione presente negli alberi della norma: una struttura di contenimento richiederebbe di duplicare il nodo condiviso in ciascun ramo che lo raggiunge, con il rischio che le copie divergano. La forma piatta coincide inoltre con quella dei file di catalogo, evitando una conversione strutturale a ogni serializzazione.

5.2.4. Node, QuestionNode e LeafNode

Ruolo: elemento atomico dell'albero. Un nodo è una domanda con due rami oppure una foglia che porta un esito.

Il dominio frontend non adotta una gerarchia di ereditarietà: QuestionNode e LeafNode sono classi indipendenti che realizzano il medesimo contratto NodeContract, e il tipo Node è definito come loro **unione discriminata** sul campo type. La scelta è motivata dal sistema dei tipi: l'unione discriminata consente al compilatore di restringere il tipo in base al valore di type, cosicché una verifica come node.type === "leaf" renda accessibile il campo outcome senza conversioni esplicite. Una classe base astratta con extends non offrirebbe questa proprietà.

NodeContract — il contratto realizzato da entrambe le classi concrete:

Attributi

- id: string : identificatore univoco del nodo all'interno dell'albero.
- type: "question" | "leaf" : discriminante dell'unione, sulla quale il compilatore restringe il tipo.

Metodi

- next(answer: boolean): string : identificatore del nodo successivo.
- verdict(): Outcome | null : esito del nodo, se ne porta uno.

QuestionNode

Attributi

- `id: string, type: "question", text: string, branches: { yes: string, no: string }`.

Metodi

- `Get: id(): string, type(): "question", text(): string, branches(): Branches` : restituiscono il corrispondente attributo.
- `next(answer: boolean): string` : restituisce l'identificatore del nodo successivo per il ramo scelto.
- `verdict(): Outcome | null` : restituisce sempre un valore nullo: un nodo domanda non porta esito.
- `toJSON(): object` : produce la forma piatta del nodo, impiegata da `DecisionTree.toJSON()`.

LeafNode

Attributi

- `id: string, type: "leaf", outcome: Outcome, text?: string`, dove `Outcome` è ristretto a `PASS`, `FAIL` e `NOT_APPLICABLE`.

Metodi

- `Get: id(): string, type(): "leaf", outcome(): Outcome, text(): string | undefined` : restituiscono il corrispondente attributo.
- `next(): string` : solleva un errore, non esistendo un successore.
- `verdict(): Outcome` : restituisce l'esito assegnato.
- `toJSON(): object` : come per `QuestionNode`.

La funzione `createNode(raw)` sceglie la classe concreta da istanziare in base al campo `type` del dato grezzo.

5.2.5. Session

Ruolo: rappresenta la valutazione nel suo insieme: quali coppie asset-requisito devono essere esaminate, quale è in corso, quale percorso è stato seguito su ciascuna e con quale esito.

Attributi

- `id: string` : identificatore della sessione.
- `savedAt: string` : istante dell'ultimo salvataggio.
- `status: "in_progress" | "completed"` : stato complessivo.

- `device`: `Device` : il dispositivo valutato, incorporato per intero nella sessione.
- `decisionTreeVersions?`: `Record<string, string>` : versione dell'albero impiegata per ciascun requisito.
- `current?`: `Current` : coppia asset-requisito in corso e nodo raggiunto.
- `evaluations`: `Evaluation[]` : una voce per ciascuna coppia del piano.

Metodi

- `Get: id(): string, savedAt(): string, status(): SessionStatus, device(): Device, decisionTreeVersions(): Record<string, string> | undefined, current(): Current | undefined, evaluations(): Evaluation[]` : restituiscono il corrispondente attributo.
- `selectEvaluation(assetId, requirementId): Session` : attiva una coppia e apre l'albero dall'inizio (UC-19).
- `syncProgress(nodeId, path): Session` : registra nodo corrente e percorso parziale senza chiudere la valutazione.
- `completeCurrent(outcome, path): Session` : registra l'esito raggiunto; la sessione passa a «completata» quando tutte le valutazioni lo sono (UC-23).
- `matchesPlan(device): boolean` : verifica che le valutazioni coprano esattamente il piano attuale del dispositivo, condizione che rende la sessione riprendibile così com'è (UC-26).
- `withSavedAt(), withEvaluations(), withDevice()` : restituiscono una sessione con il rispettivo campo aggiornato.
- `toJSON(): object` : produce la rappresentazione serializzabile della sessione, con il dispositivo a sua volta serializzato; è la forma scritta nel file di sessione.
- `start(device, id, savedAt): Session` : costruisce una sessione nuova a partire dal piano del dispositivo, generando una valutazione in stato «non valutato» per ciascuna coppia asset-requisito e attivando la prima. Se il piano è vuoto la sessione nasce già completata (UC-19).
- `parse(raw): Session` : ricostruisce una sessione dal contenuto di un file caricato dall'utente, validandolo con `SessionSchema` (UC-26).

Responsabilità: custodire lo stato complessivo della valutazione e le transizioni ammesse fra i suoi stati, garantendo che ogni modifica produca una sessione coerente.

La sessione **incorpora** il dispositivo anziché referenziarlo per identificatore: il file di sessione è così autosufficiente e può essere ripreso senza disporre altrove del dispositivo. `matchesPlan()` verifica al momento della ripresa che il dispositivo incorporato produca lo stesso piano, condizione senza la quale la sessione non sarebbe più significativa.

Le strutture `Evaluation`, `Current` e `PathStep` restano dati puri, definiti come tipi derivati dai rispettivi schemi e non come classi: non possiedono comportamento proprio né invarianti da proteggere oltre alla forma, che è già garantita dalla validazione.

5.3. Dominio backend

Le entità del backend sono realizzate come `dataclass` immutabili (`frozen=True`). Le entità dotate di validazione — `Device`, `Asset`, `QuestionNode` e `LeafNode` — dichiarano i campi come privati secondo la convenzione del trattino basso ed espongono proprietà di sola lettura omonime; le strutture prive di invarianti da proteggere dichiarano invece campi pubblici. La costruzione a partire da dati esterni avviene tramite metodi di classe `create()`, che validano i campi obbligatori e sollevano l'eccezione corrispondente in caso di dato non conforme.

5.3.1. AssetType

Ruolo: enumerazione delle quattro tipologie di asset previste dalla norma EN 18031.

Valori

- `NETWORK`, `SECURITY`, `PRIVACY`, `FINANCIAL`, con i rispettivi valori testuali `"network"`, `"security"`, `"privacy"` e `"financial"`.

Metodi

- `from_string(value): AssetType (di classe)` : converte il valore testuale ricevuto dal client nel membro corrispondente, sollevando `ValueError` se non riconosciuto.

Derivando da `str`, i membri sono direttamente serializzabili senza conversione esplicita.

5.3.2. Device

Ruolo: rappresenta il dispositivo sottoposto a valutazione e aggrega i propri asset.

Attributi

- `_id: str` : identificatore univoco.
- `_name: str` : denominazione del dispositivo.
- `_operating_system: str` : sistema operativo.
- `_description: str` : descrizione testuale.
- `_assets: list[Asset]` : asset associati.

Metodi

- Proprietà di sola lettura: `id`, `name`, `operating_system`, `description`, `assets` : restituiscono il corrispondente attributo.
- `create(data): Device (di classe)` : valida la presenza dei campi obbligatori, risolve l'identificatore rispettandolo se fornito o generandolo con `uuid4` altrimenti, e costruisce

l'entità con elenco degli asset vuoto. Solleva `InvalidDeviceDataError` in caso di dato non conforme.

Responsabilità: costituire il punto unico di validazione dei metadati del dispositivo, condiviso dalla creazione manuale e dall'importazione.

5.3.3. Asset

Ruolo: rappresenta un elemento del dispositivo soggetto a valutazione.

Attributi

- `_id: str`: identificatore univoco.
- `_name: str`: denominazione dell'asset.
- `_type: AssetType`: tipologia funzionale.
- `_description: str`: descrizione testuale.
- `_sensitive: bool`: indica se l'asset tratta dati sensibili.
- `_requirements: list[str]`: codici dei requisiti applicabili.

Metodi

- Proprietà di sola lettura: `id`, `name`, `type`, `description`, `sensitive`, `requirements`: restituiscono il corrispondente attributo. Le ultime due restituiscono una copia dell'elenco, a tutela dell'immutabilità.
- `create(data): Asset` (di classe): valida i campi e converte la tipologia testuale tramite `AssetType.from_string()`. Solleva `InvalidAssetDataError` in caso di dato non conforme.

5.3.4. NodeOutcome

Ruolo: enumerazione dei tre esiti che una foglia può portare.

Valori

- `PASS`, `FAIL`, `NOT_APPLICABLE`.

Metodi

- `from_string(value): NodeOutcome` (di classe): converte il valore testuale letto dai file di catalogo, simmetricamente ad `AssetType`.

5.3.5. Branches

Ruolo: coppia dei rami di un nodo domanda.

Attributi

- `yes: str`: identificatore del nodo successivo per la risposta affermativa.

- `no: str`: identificatore del nodo successivo per la risposta negativa.

5.3.6. Node, QuestionNode e LeafNode

Ruolo: elemento atomico dell'albero. Un nodo è una domanda con due rami oppure una foglia che porta un esito.

A differenza del frontend, il backend adotta una gerarchia di ereditarietà: `Node` è una classe astratta e le due classi concrete la specializzano.

Node (astratta)

Metodi

- `id`: proprietà astratta, identificatore del nodo.
- `next(answer): str`: astratto, identificatore del nodo successivo.
- `verdict(): NodeOutcome | None`: astratto, esito del nodo se ne porta uno.

QuestionNode

Attributi

- `_id: str, _text: str, _branches: Branches`, oltre alla tipologia `type`, che vale "question".

Metodi

- Proprietà di sola lettura: `id, type, text, branches`.
- `next(answer): str`: restituisce l'identificatore del ramo scelto.
- `verdict(): NodeOutcome | None`: restituisce sempre un valore nullo.

LeafNode

Attributi

- `_id: str, _outcome: NodeOutcome, _text: str | None`, oltre alla tipologia `type`, che vale "leaf".

Metodi

- Proprietà di sola lettura: `id, type, outcome, text`.
- `next(answer): str`: solleva un errore, non esistendo un successore.
- `verdict(): NodeOutcome | None`: restituisce l'esito assegnato.

5.3.7. DecisionTree

Ruolo: rappresenta l'albero decisionale che guida la valutazione di un requisito.

Attributi (pubblici)

- `requirement_id: str` : codice del requisito.
- `requirement_name: str` : denominazione estesa.
- `root_node: str` : identificatore del nodo radice.
- `nodes: list[Node]` : collezione dei nodi in forma piatta.
- `version: str | None` : versione dell'albero.
- `applies_to: list[str]` : tipologie di asset alle quali il requisito si applica.
- `dependencies: list[str]` : requisiti dai quali questo dipende.

Metodi

- `get_node(node_id): Node` : reperisce un nodo per identificatore, sollevando `KeyError` se assente.

5.3.8. PathStep, Evaluation, Current e Session

Ruolo: rappresentano il file di sessione. Sono definite per completezza del modello, ma nessuna rotta le espone: la sessione è gestita interamente dal client.

Tutti i campi sono pubblici, trattandosi di strutture dati prive di comportamento e di invarianti da proteggere.

PathStep

- `node_id: str` : nodo attraversato.
- `answer: Answer` : risposta fornita, "yes" o "no".

Evaluation

- `asset_id: str, requirement_id: str` : coppia valutata.
- `status: EvaluationStatus` : "not_evaluated", "in_progress" o "completed".
- `outcome: NodeOutcome | None` : esito raggiunto.
- `justification: str | None` : giustificazione testuale.
- `path: list[PathStep]` : percorso seguito.

Current

- `asset_id: str, requirement_id: str, node_id: str` : coppia in corso e nodo raggiunto.

Session

- `id: str, saved_at: str` : identificatore e istante di salvataggio.
- `status: SessionStatus` : "in_progress" o "completed".

- **device:** Device : dispositivo valutato, incorporato per intero.
- **evaluations:** list[Evaluation] : una voce per ciascuna coppia del piano.
- **decision_tree_versions:** dict[str, str] : versione dell'albero impiegata per requisito.
- **current:** Current | None : coppia in corso.

5.4. Corrispondenza fra i due domini

Le due realizzazioni descrivono i medesimi concetti, ma differiscono in tre punti, ciascuno per una ragione precisa.

Rappresentazione dei nodi. Il backend adotta una gerarchia di ereditarietà a partire da una classe astratta; il frontend un'unione discriminata su un contratto comune. La differenza discende dai rispettivi sistemi dei tipi: il restringimento del tipo su un campo discriminante è una proprietà di TypeScript senza equivalente diretto in Python, dove l'ereditarietà con metodi astratti è la costruzione idiomatica.

Presenza della sessione. Session esiste su entrambi i lati, ma solo il frontend la utilizza. La definizione lato server documenta la forma del file di sessione senza che alcuna rotta la impieghi, coerentemente con il criterio di possesso del dato: la sessione appartiene all'utente, non al sistema.

Estensione del comportamento. Le entità del frontend espongono più comportamento perché è nel client che risiede la logica di valutazione. Le entità del backend si limitano alla validazione in ingresso e alla navigazione dell'albero, che sono le sole responsabilità del server.

Le due realizzazioni non sono generate l'una dall'altra né condividono codice: la coerenza fra esse è garantita dal contratto degli endpoint, che costituisce l'unica fonte di verità condivisa.

5.5. Vincoli e regole di integrità

Le seguenti regole definiscono l'integrità del dominio e sono verificate dal backend in fase di validazione di un albero, all'atto del caricamento e dell'importazione.

- Ogni nodo possiede un identificatore univoco all'interno del medesimo decision tree.
- Il nodo radice dichiarato deve esistere fra i nodi dell'albero.
- Ogni ramo di un nodo domanda deve puntare a un nodo esistente: non sono ammessi riferimenti pendenti.
- Un nodo domanda deve possedere testo non vuoto ed entrambi i rami; un nodo foglia deve possedere un esito fra PASS, FAIL e NOT_APPLICABLE.
- Il codice del requisito deve rispettare il formato previsto.

- L'albero non deve contenere cicli.

6. Tracciamento dei requisiti

In questa sezione è riportato lo stato di copertura dei requisiti funzionali individuati nell'[Analisi dei Requisiti](#), rispetto a quanto realizzato nel prodotto descritto dal presente documento. Per ciascun requisito sono riportati il codice identificativo, la descrizione e lo stato.

Gli stati impiegati sono due:

- **Soddisfatto:** il requisito è realizzato nel prodotto ed è verificabile attraverso l'uso dell'applicazione.
- **Non soddisfatto:** il requisito non è realizzato nella versione corrente del prodotto.

I codici mantengono la classificazione per priorità adottata in sede di analisi: obbligatori (RF-Ob), desiderabili (RF-D) e opzionali (RF-Op). Le sezioni che seguono ne riportano lo stato distintamente per ciascuna categoria.

6.1. Requisiti funzionali obbligatori

Costituiscono il nucleo funzionale imprescindibile del prodotto: il loro soddisfacimento è condizione necessaria affinché l'applicazione risponda ai bisogni primari espressi dalla proponente.

Codice	Descrizione	Stato
RF-Ob01	Il sistema deve permettere l'inserimento di un nuovo dispositivo all'interno della piattaforma.	Soddisfatto
RF-Ob02	Il sistema deve permettere l'importazione di un dispositivo tramite un file di configurazione in formato JSON o CSV.	Soddisfatto
RF-Ob03	Il sistema deve permettere la selezione del file sorgente per l'importazione del dispositivo.	Soddisfatto
RF-Ob04	Il sistema deve supportare la selezione di un file in formato JSON come sorgente per l'importazione del dispositivo.	Soddisfatto
RF-Ob05	Il sistema deve supportare la selezione di un file in formato CSV come sorgente per l'importazione del dispositivo.	Soddisfatto
RF-Ob06	Il sistema deve verificare la validità strutturale e la conformità del file di configurazione caricato.	Soddisfatto
RF-Ob07	Il sistema deve bloccare l'importazione e mostrare un messaggio di errore se il file ha un formato non valido.	Soddisfatto
RF-Ob08	Il sistema deve permettere la creazione manuale di un nuovo dispositivo.	Soddisfatto
RF-Ob09	Il sistema deve richiedere l'inserimento dei dati identificativi del dispositivo durante la creazione manuale.	Soddisfatto
RF-Ob10	Il sistema deve permettere l'inserimento del nome identificativo del dispositivo.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob11	Il sistema deve permettere l'inserimento del sistema operativo del dispositivo.	Soddisfatto
RF-Ob12	Il sistema deve permettere l'inserimento di una descrizione testuale del dispositivo.	Soddisfatto
RF-Ob13	Il sistema deve validare i dati inseriti nei form e mostrare un errore in caso di campi vuoti o non conformi.	Soddisfatto
RF-Ob14	Il sistema deve permettere la visualizzazione delle informazioni e dei dati relativi al dispositivo.	Soddisfatto
RF-Ob15	Il sistema deve mostrare in dettaglio il nome del dispositivo registrato.	Soddisfatto
RF-Ob16	Il sistema deve mostrare in dettaglio il sistema operativo del dispositivo registrato.	Soddisfatto
RF-Ob17	Il sistema deve mostrare in dettaglio la descrizione del dispositivo registrato.	Soddisfatto
RF-Ob18	Il sistema deve calcolare e mostrare lo stato aggregato di valutazione del dispositivo (non valutato, PASS, FAIL).	Soddisfatto
RF-Ob19	Il sistema deve permettere l'esportazione di tutti i dati di un dispositivo in formato JSON o CSV.	Soddisfatto
RF-Ob20	Il sistema deve consentire l'esportazione dei dati del dispositivo e degli asset associati in formato JSON.	Soddisfatto
RF-Ob21	Il sistema deve consentire l'esportazione dei dati del dispositivo e degli asset associati in formato CSV.	Soddisfatto
RF-Ob22	Il sistema deve permettere l'eliminazione definitiva di un dispositivo dal sistema.	Soddisfatto
RF-Ob23	Il sistema deve consentire l'eliminazione diretta di un dispositivo senza effettuare il backup dei dati.	Soddisfatto
RF-Ob24	Il sistema deve consentire l'eliminazione del dispositivo previa esportazione automatica di backup dei dati.	Soddisfatto
RF-Ob25	Il sistema deve permettere l'inserimento di un nuovo asset all'interno di un dispositivo.	Soddisfatto
RF-Ob26	Il sistema deve richiedere la compilazione dei dati dell'asset nel form di creazione.	Soddisfatto
RF-Ob27	Il sistema deve permettere l'inserimento del nome dell'asset nel form di creazione.	Soddisfatto
RF-Ob28	Il sistema deve permettere la selezione del tipo di asset tra Network, Security, Privacy e Financial.	Soddisfatto
RF-Ob29	Il sistema deve permettere l'inserimento della descrizione dell'asset nel form di creazione.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob30	Il sistema deve permettere di impostare la sensibilità dell'asset.	Soddisfatto
RF-Ob31	Il sistema deve permettere la visualizzazione della lista degli asset associati ad un determinato dispositivo.	Soddisfatto
RF-Ob32	Il sistema deve mostrare le informazioni essenziali del singolo asset all'interno della lista.	Soddisfatto
RF-Ob33	Il sistema deve mostrare il nome del singolo asset all'interno della lista.	Soddisfatto
RF-Ob34	Il sistema deve mostrare il tipo del singolo asset all'interno della lista.	Soddisfatto
RF-Ob35	Il sistema deve mostrare lo stato di valutazione del singolo asset all'interno della lista.	Soddisfatto
RF-Ob36	Il sistema deve permettere la visualizzazione in dettaglio di tutte le informazioni di un singolo asset selezionato.	Soddisfatto
RF-Ob37	Il sistema deve mostrare nel dettaglio il nome dell'asset selezionato.	Soddisfatto
RF-Ob38	Il sistema deve mostrare nel dettaglio il tipo dell'asset selezionato.	Soddisfatto
RF-Ob39	Il sistema deve mostrare nel dettaglio la descrizione dell'asset selezionato.	Soddisfatto
RF-Ob40	Il sistema deve mostrare nel dettaglio il livello di sensibilità dell'asset selezionato.	Soddisfatto
RF-Ob41	Il sistema deve mostrare lo stato complessivo di valutazione dell'asset selezionato.	Soddisfatto
RF-Ob42	Il sistema deve mostrare la lista dei requisiti da valutare associati all'asset.	Soddisfatto
RF-Ob43	Il sistema deve mostrare il codice identificativo e lo stato di valutazione di ogni requisito nella lista.	Soddisfatto
RF-Ob44	Il sistema deve permettere l'eliminazione definitiva di un asset da un dispositivo.	Soddisfatto
RF-Ob45	Il sistema deve permettere l'esecuzione di una sessione di valutazione	Soddisfatto
RF-Ob46	Il sistema deve mostrare una dashboard di valutazione con la lista degli asset, il loro stato, e il progresso della sessione in tempo reale.	Soddisfatto
RF-Ob47	Il sistema deve consentire la selezione e l'avvio della valutazione dei requisiti di un singolo asset.	Soddisfatto
RF-Ob48	Il sistema deve mostrare il nome, il tipo, la descrizione, la sensibilità e lo stato di valutazione dell'asset selezionato per la valutazione.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob49	Il sistema deve mostrare la lista dei requisiti associati all'asset in valutazione.	Soddisfatto
RF-Ob50	Il sistema deve mostrare, per ciascun requisito nella lista, il codice e lo stato di valutazione.	Soddisfatto
RF-Ob51	Il sistema deve mostrare il codice e il nome del requisito selezionato prima dell'avvio dell'esecuzione del decision tree.	Soddisfatto
RF-Ob52	Il sistema deve mostrare le dipendenze del requisito selezionato e il loro stato di valutazione prima dell'esecuzione.	Soddisfatto
RF-Ob53	Il sistema deve guidare l'utente eseguendo il decision tree associato al requisito selezionato.	Soddisfatto
RF-Ob54	Il sistema deve mostrare il codice univoco e il testo della domanda del nodo corrente dell'albero.	Soddisfatto
RF-Ob55	Il sistema deve registrare la risposta dell'utente avanzandone il percorso sul grafo.	Soddisfatto
RF-Ob56	Il sistema deve gestire la risposta affermativa («Yes») spostando il flusso sul relativo ramo.	Soddisfatto
RF-Ob57	Il sistema deve gestire la risposta negativa («No») spostando il flusso sul relativo ramo.	Soddisfatto
RF-Ob58	Il sistema deve visualizzare a schermo il grafo completo del decision tree durante l'esecuzione.	Soddisfatto
RF-Ob59	Il sistema deve evidenziare graficamente nel grafo il nodo corrente e il percorso già intrapreso.	Soddisfatto
RF-Ob60	Il sistema deve mostrare un nodo foglia al termine del percorso con l'esito (PASS, FAIL, NOT APPLICABLE).	Soddisfatto
RF-Ob61	Il sistema deve generare un file JSON contenente lo stato della sessione di valutazione per il download.	Soddisfatto
RF-Ob62	Il sistema deve permettere il caricamento di un file di sessione per riprendere un test interrotto.	Soddisfatto
RF-Ob63	Il sistema deve mostrare una schermata finale con il riepilogo complessivo di tutti gli esiti del test.	Soddisfatto
RF-Ob64	Il sistema deve mostrare per ogni asset la lista dei requisiti completati e il percorso logico seguito.	Soddisfatto
RF-Ob65	Il sistema deve mostrare la sequenza ordinata di domande e risposte fornite per un requisito completato.	Soddisfatto
RF-Ob66	Il sistema deve mostrare l'elenco dei decision tree disponibili.	Soddisfatto
RF-Ob67	Il sistema deve mostrare l'ID e il nome del requisito per ogni decision tree in elenco.	Soddisfatto

Codice	Descrizione	Stato
RF-Ob68	Il sistema deve permettere la visualizzazione in dettaglio di un decision tree esistente, mostrandone l'identificativo e il nome del requisito associato.	Soddisfatto
RF-Ob69	Il sistema deve permettere all'utente di uscire anticipatamente da una sessione di valutazione in corso.	Soddisfatto
RF-Ob70	Il sistema deve mostrare il riepilogo degli esiti per ogni singolo asset al termine del test.	Soddisfatto
RF-Ob71	Il sistema deve permettere il salvataggio della sessione di valutazione in corso, generando un file con lo stato della sessione.	Soddisfatto
RF-Ob72	Il sistema deve visualizzare il grafo del decision tree nel dettaglio, mostrando nodi interni, nodi foglia con esito e collegamenti fra nodi.	Soddisfatto
RF-Ob73	Il sistema deve mostrare i nodi interni del grafo del decision tree, con il relativo codice univoco e il testo della domanda.	Soddisfatto
RF-Ob74	Il sistema deve mostrare i nodi foglia del grafo del decision tree, con l'esito associato (PASS, FAIL, NOT APPLICABLE).	Soddisfatto
RF-Ob75	Il sistema deve mostrare i collegamenti fra i nodi del grafo del decision tree, con l'etichetta Yes/No associata a ciascun ramo.	Soddisfatto
RF-Ob76	Il sistema deve mostrare le dipendenze del decision tree, elencando i requisiti da cui esso dipende con il relativo codice.	Soddisfatto
RF-Ob77	Il sistema deve permettere l'esportazione di un file di un decision tree in formato JSON o CSV.	Soddisfatto
RF-Ob78	Il sistema deve consentire l'esportazione di un decision tree in formato JSON.	Soddisfatto
RF-Ob79	Il sistema deve consentire l'esportazione di un decision tree in formato CSV.	Soddisfatto
RF-Ob80	Il sistema deve generare un report di conformità finale contenente, per ogni coppia asset-requisito, l'esito del requisito, l'esito aggregato del decision tree e il percorso logico seguito.	Soddisfatto

Copertura: 80 requisiti soddisfatti su 80 (100%).

6.2. Requisiti funzionali desiderabili

Introducono funzionalità a valore aggiunto che migliorano la fluidità del flusso operativo, senza costituire condizione bloccante per l'utilizzo del prodotto.

Codice	Descrizione	Stato
RF-D01	Il sistema deve permettere l'annullamento della procedura di inserimento di un dispositivo, ripristinando lo stato precedente.	Soddisfatto

Codice	Descrizione	Stato
RF-D02	Il sistema deve permettere l'annullamento della procedura di modifica di un dispositivo, scartando i dati inseriti e mantenendo invariati quelli preesistenti.	Soddisfatto
RF-D03	Il sistema deve permettere l'annullamento della procedura di eliminazione di un dispositivo durante la fase di richiesta di conferma.	Soddisfatto
RF-D04	Il sistema deve permettere l'annullamento della procedura di inserimento di un asset, ripristinando lo stato precedente.	Soddisfatto
RF-D05	Il sistema deve permettere l'annullamento della procedura di modifica di un asset, scartando le modifiche non salvate.	Soddisfatto
RF-D06	Il sistema deve permettere l'annullamento della procedura di eliminazione di un asset durante la fase di richiesta di conferma.	Soddisfatto
RF-D07	Il sistema deve permettere la navigazione al nodo precedente del decision tree, mostrando la risposta già fornita senza invalidare le risposte successive.	Soddisfatto
RF-D08	Il sistema deve permettere il salvataggio intermedio dello stato della sessione di valutazione.	Soddisfatto
RF-D09	Il sistema deve permettere all'utente di navigare verso il nodo successivo precedentemente già risposto durante l'esecuzione del decision tree.	Soddisfatto
RF-D10	Il sistema deve permettere la modifica della risposta a un nodo già risposto, invalidando le risposte successive al nodo corrente.	Soddisfatto
RF-D11	Il sistema deve permettere la modifica delle informazioni anagrafiche di un dispositivo esistente.	Soddisfatto
RF-D12	Il sistema deve consentire la modifica del nome del dispositivo.	Soddisfatto
RF-D13	Il sistema deve consentire la modifica del sistema operativo del dispositivo.	Soddisfatto
RF-D14	Il sistema deve consentire la modifica della descrizione del dispositivo.	Soddisfatto
RF-D15	Il sistema deve permettere la modifica delle informazioni di un asset esistente.	Soddisfatto
RF-D16	Il sistema deve consentire la modifica del nome dell'asset.	Soddisfatto
RF-D17	Il sistema deve consentire la modifica del tipo dell'asset tramite opzioni predefinite.	Soddisfatto
RF-D18	Il sistema deve consentire la modifica della descrizione dell'asset.	Soddisfatto
RF-D19	Il sistema deve consentire la modifica della sensibilità dell'asset.	Soddisfatto
RF-D20	Il sistema deve permettere l'importazione e la validazione strutturale di un nuovo decision tree da file.	Soddisfatto

Codice	Descrizione	Stato
RF-D21	Il sistema deve permettere la selezione del file sorgente per l'importazione di un decision tree.	Soddisfatto
RF-D22	Il sistema deve supportare l'importazione di un decision tree da file in formato JSON.	Soddisfatto
RF-D23	Il sistema deve supportare l'importazione di un decision tree da file in formato CSV.	Soddisfatto

Copertura: 23 requisiti soddisfatti su 23 (100%).

6.3. Requisiti funzionali opzionali

Estendono la piattaforma con funzionalità ausiliarie e di raffinamento, gradite ma non essenziali ai fini della validazione dell'applicazione.

Codice	Descrizione	Stato
RF-Op01	Il sistema deve mostrare la notifica dell'avvenuto salvataggio intermedio della sessione di valutazione.	Non soddisfatto
RF-Op02	Il sistema deve permettere l'aggiunta manuale di una dipendenza tra requisiti all'interno di un decision tree.	Non soddisfatto
RF-Op03	Il sistema deve permettere la rimozione di una dipendenza tra requisiti da un decision tree.	Non soddisfatto
RF-Op04	Il sistema deve bloccare l'aggiunta e notificare l'utente se il requisito selezionato crea una dipendenza circolare.	Non soddisfatto
RF-Op05	Il sistema deve permettere l'aggiunta di un nuovo nodo all'interno di un decision tree.	Non soddisfatto
RF-Op06	Il sistema deve permettere l'inserimento di un codice univoco per il nuovo nodo.	Non soddisfatto
RF-Op07	Il sistema deve permettere l'inserimento del testo della domanda del nuovo nodo.	Non soddisfatto
RF-Op08	Il sistema deve permettere l'eliminazione di un nodo esistente da un decision tree.	Non soddisfatto
RF-Op09	Il sistema deve impedire la creazione di collegamenti duplicati mostrando una notifica di errore.	Non soddisfatto
RF-Op10	Il sistema deve validare la struttura dell'albero modificato secondo i vincoli di consistenza predefiniti.	Non soddisfatto
RF-Op11	Il sistema deve impedire il salvataggio e mostrare un errore se l'albero non è binario o mancano foglie PASS/FAIL.	Non soddisfatto
RF-Op12	Il sistema deve impedire l'eliminazione del nodo radice di un decision tree mostrando un errore.	Non soddisfatto
RF-Op13	Il sistema deve consentire l'eliminazione definitiva di un decision tree.	Soddisfatto

Codice	Descrizione	Stato
RF-Op14	Il sistema deve bloccare l'inserimento e mostrare un messaggio di errore se il codice del nodo è già presente nel decision tree.	Non soddisfatto
RF-Op15	Il sistema deve permettere all'utente di assegnare un esito (PASS, FAIL o NOT APPLICABLE) ai rami non collegati di un nodo appena aggiunto o modificato nel decision tree, trasformandoli in nodi foglia.	Non soddisfatto
RF-Op16	Il sistema deve assegnare l'esito PASS al ramo non collegato selezionato dall'utente, creando un nodo foglia PASS.	Non soddisfatto
RF-Op17	Il sistema deve assegnare l'esito FAIL al ramo non collegato selezionato dall'utente, creando un nodo foglia FAIL.	Non soddisfatto
RF-Op18	Il sistema deve assegnare l'esito NOT APPLICABLE al ramo non collegato selezionato dall'utente, creando un nodo foglia NOT APPLICABLE.	Non soddisfatto
RF-Op19	Il sistema deve permettere l'annullamento delle modifiche effettuate su un decision tree, ripristinando lo stato iniziale del grafo.	Non soddisfatto
RF-Op20	Il sistema deve permettere la modifica della destinazione di un collegamento tra nodi (Yes/No).	Non soddisfatto
RF-Op21	Il sistema deve permettere la modifica strutturale di un decision tree esistente.	Non soddisfatto
RF-Op22	Il sistema deve consentire il download del report di conformità in formato PDF.	Soddisfatto
RF-Op23	Il sistema deve consentire il download del report di conformità in formato JSON.	Non soddisfatto
RF-Op24	Il sistema deve consentire il download del report di conformità in formato CSV.	Non soddisfatto
RF-Op25	Il sistema deve mostrare la giustificazione testuale del risultato raggiunto al termine dell'esecuzione del decision tree.	Non soddisfatto
RF-Op26	Il sistema deve permettere l'inserimento di una giustificazione testuale per l'esito della coppia asset-requisito al termine dell'esecuzione del decision tree.	Non soddisfatto

Copertura: 2 requisiti soddisfatti su 26 (7.7%).

I requisiti soddisfatti della categoria sono RF-Op13, relativo all'eliminazione di un decision tree, e RF-Op22, relativo al download del report in formato PDF. Gli altri requisiti opzionali riguardano in prevalenza la modifica strutturale dei decision tree, l'esportazione del report negli altri formati e funzionalità accessorie di consultazione. La loro realizzazione è

subordinata al completamento dei requisiti obbligatori e desiderabili e alla disponibilità di tempo residuo, secondo quanto stabilito in sede di pianificazione.

7. Requisiti di sistema

Il prodotto è distribuito come applicazione web locale containerizzata. I requisiti riportati in questa sezione si riferiscono pertanto alla macchina che ospita i container e al browser impiegato per accedere all'interfaccia; non è previsto alcun server remoto né alcuna infrastruttura condivisa.

Si distinguono due scenari con esigenze differenti: la costruzione delle immagini, che avviene una sola volta e richiede il collegamento a Internet, e l'esecuzione ordinaria, che avviene interamente in locale.

7.1. Requisiti hardware

I valori riportati sono indicativi e possono variare in funzione del sistema operativo, della versione di Docker e della quantità di dati presenti nel catalogo.

Risorsa	Rilevato	Consigliato
Memoria occupata dai container	42 MiB	—
Memoria complessiva della macchina	—	4 GB
Spazio per le immagini	288 MB	—
Spazio complessivo su disco	—	2 GB
Catalogo dei decision tree	40 KB	—

L'occupazione di memoria dei due container è di 22 MiB per il backend e 20 MiB per il frontend. Nel test effettuato non sono state osservate variazioni significative durante l'uso. La ragione è che il servizio di frontend si limita a servire file statici tramite Nginx, mentre l'applicazione vera e propria è eseguita dal browser dell'utente. Il consumo di risorse significativo è pertanto quello del browser, non quello dei container, ed è il motivo per cui la memoria complessiva consigliata è di 4 GB in modo da essere sufficiente a ospitare il motore di containerizzazione, il browser e il sistema operativo. Le immagini prodotte occupano complessivamente 288 MB, di cui 212 MB per il backend e 76,1 MB per il frontend. Lo spazio consigliato di 2 GB tiene conto dei livelli temporanei generati durante la costruzione delle immagini, in particolare le dipendenze di sviluppo del frontend, assenti dall'immagine finale grazie alla costruzione in due fasi e del margine necessario alla crescita del catalogo dei decision tree, che l'utente può ampliare tramite importazione.

Il catalogo dei decision tree occupa allo stato attuale 40 KB, essendo costituito da documenti JSON di dimensioni ridotte. La sua crescita è lineare nel numero di alberi importati e non pone vincoli di rilievo.

Nessun limite di risorse è dichiarato nel file di orchestrazione: i container utilizzano quanto reso disponibile dal motore di containerizzazione.

7.2. Requisiti software

Componente	Versione	Note
Sistema operativo	—	Qualsiasi sistema in grado di eseguire Docker: distribuzioni Linux, macOS, Windows.
Docker Engine	20.10 o successiva	Necessario per la costruzione e l'esecuzione dei container. Su macOS e Windows è fornito da Docker Desktop.
Docker Compose	v2	Impiegato per l'orchestrazione dei due servizi tramite il file <code>docker-compose.yml</code> .
Browser web	—	Necessario per accedere all'interfaccia. I browser supportati sono indicati di seguito.

Sulla macchina ospite non è richiesta l'installazione né di Python né di Node.js: entrambi gli ambienti di esecuzione sono contenuti nelle rispettive immagini. Il backend è costruito a partire da `python:3.12-slim`, il frontend è compilato con `node:22-alpine` e distribuito su `nginx:1.27-alpine`; l'ambiente di compilazione non è presente nell'immagine finale.

Il requisito di vincolo RV-Ob02 stabilisce la compatibilità con le ultime due versioni principali di Google Chrome, Mozilla Firefox, Apple Safari e Microsoft Edge. Il prodotto è realizzato con tecnologie web standard e non impiega funzionalità sperimentali o specifiche di un singolo motore di rendering.

7.3. Requisiti di rete

L'esecuzione del prodotto non richiede alcun collegamento a Internet. Client e server comunicano sulla rete privata creata da Docker Compose, e l'interfaccia è raggiungibile dall'ospite all'indirizzo `http://localhost:8080`.

Le porte seguenti devono essere disponibili sulla macchina ospite:

Porta	Servizio	Utilizzo
8080	frontend	Accesso all'interfaccia utente. Corrisponde alla porta 80 del container, sulla quale è in ascolto Nginx.
5000	backend	Accesso diretto all'API REST. Non necessaria all'uso ordinario, poiché le richieste dell'interfaccia sono inoltrate al backend da Nginx sulla rete privata; è esposta per finalità di collaudo e diagnosi.

Il collegamento a Internet è necessario unicamente in fase di **costruzione** delle immagini, per il recupero delle immagini base e delle dipendenze applicative dai rispettivi registri. Una volta costruite, le immagini possono essere eseguite su una macchina isolata dalla rete esterna.

L'assenza di dipendenze da servizi remoti è coerente con la natura del prodotto: i dati dell'utente non transitano né permangono su alcun archivio esterno, e lo scambio di dispositivi, sessioni e decision tree fra installazioni avviene mediante esportazione e importazione di file.